ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837



Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο

Διδάσκων: Δρ. Γ. Χαμόδρακας

Ομάδα

Όλγα Πασβάνη
Αριθμός Μητρώου: 1115202100146

Ελένη Δημηρώση
Αριθμός Μητρώου: 1115202100030

Εισαγωγή

Η ανάπτυξη σύγχρονων διαδικτυακών εφαρμογών διαχείρισης εκδηλώσεων απαιτεί τον συνδυασμό πολλών διαφορετικών λειτουργιών, όπως η διαχείριση χρηστών και ρόλων, η αναζήτηση εκδηλώσεων, η πραγματοποίηση κρατήσεων, η ασφαλής διαχείριση ταυτόχρονης πρόσβασης, η επικοινωνία μεταξύ χρηστών και η παροχή προσωποποιημένων υπηρεσιών. Ιδιαίτερα σε ένα σύστημα ηλεκτρονικών κρατήσεων, η ορθότητα και η συνέπεια των δεδομένων είναι κρίσιμες, καθώς πολλοί χρήστες είναι δυνατόν να αλληλεπιδρούν ταυτόχρονα με τις ίδιες διαθέσιμες θέσεις.

Σκοπός της παρούσας εργασίας είναι η σχεδίαση και ανάπτυξη μιας ολοκληρωμένης web εφαρμογής διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων εισιτηρίων/θέσεων, βασισμένης στην αρχιτεκτονική web browser / web server. Η εφαρμογή καλύπτει ολόκληρο τον κύκλο ζωής μιας εκδήλωσης, από τη δημιουργία και δημοσίευσή της έως την αναζήτηση, την πραγματοποίηση κρατήσεων, την επικοινωνία μεταξύ διοργανωτών και συμμετεχόντων, την εξαγωγή δεδομένων και την παραγωγή προσωποποιημένων συστάσεων.

Το σύστημα υποστηρίζει τέσσερις διαφορετικούς ρόλους χρηστών: Επισκέπτη, Συμμετέχοντα, Διοργανωτή και Διαχειριστή. Κάθε ρόλος διαθέτει συγκεκριμένες δυνατότητες και περιορισμούς, οι οποίοι εφαρμόζονται τόσο στο επίπεδο του γραφικού περιβάλλοντος όσο και στο backend, μέσω μηχανισμού εξουσιοδότησης βάσει ρόλων.

Η υλοποίηση ακολουθεί αρχιτεκτονική client-server με σαφή διαχωρισμό των επιμέρους επιπέδων. Το backend αναπτύχθηκε σε Python με χρήση του FastAPI και παρέχει REST API για όλες τις λειτουργίες της εφαρμογής. Η πρόσβαση στη σχεσιακή βάση δεδομένων πραγματοποιείται μέσω SQLAlchemy ORM, ενώ ως κύριο σύστημα διαχείρισης βάσης δεδομένων χρησιμοποιείται PostgreSQL και η εξέλιξη του σχήματος ελέγχεται μέσω Alembic migrations. Το frontend υλοποιήθηκε ως Single Page Application με React και TypeScript και επικοινωνεί αποκλειστικά με το REST API. Για την αυθεντικοποίηση χρησιμοποιούνται JSON Web Tokens (JWT), ενώ η εξουσιοδότηση υλοποιείται με Role-Based Access Control (RBAC). Στο περιβάλλον Docker Compose, η επικοινωνία του χρήστη με την εφαρμογή πραγματοποιείται μέσω HTTPS με χρήση Caddy ως reverse proxy.

Ιδιαίτερη έμφαση δόθηκε στην ακεραιότητα των δεδομένων κατά την ταυτόχρονη πραγματοποίηση κρατήσεων, στην ασφάλεια της εφαρμογής και στην υλοποίηση του συστήματος προσωποποιημένων συστάσεων. Για τον τελευταίο μηχανισμό αναπτύχθηκε από το μηδέν αλγόριθμος Biased Matrix Factorization, ο οποίος αξιοποιεί έμμεση ανατροφοδότηση από προβολές και κρατήσεις χρηστών.

Στα επόμενα κεφάλαια παρουσιάζονται διαδοχικά η αρχιτεκτονική του συστήματος, η σχεδίαση της βάσης δεδομένων, οι ρόλοι χρηστών και οι μηχανισμοί αυθεντικοποίησης, το REST API, η διαχείριση εκδηλώσεων, η αναζήτηση και η σελιδοποίηση, ο μηχανισμός κρατήσεων και ταυτόχρονης πρόσβασης, η ενσωμάτωση χαρτών OpenStreetMap, το σύστημα μηνυμάτων, η εξαγωγή δεδομένων σε XML και JSON, ο αλγόριθμος συστάσεων, το frontend, οι μηχανισμοί ασφάλειας, οι διαδικασίες εγκατάστασης και εκτέλεσης και, τέλος, οι βασικές παραδοχές και σχεδιαστικές αποφάσεις της υλοποίησης.

Πίνακας Περιεχομένων

Εισαγωγή 2
Πίνακας Περιεχομένων 3
Πίνακας εικόνων 5
Πίνακας πινάκων 6
Ορισμοί, Ακρωνύμια και Συντομογραφίες 7
1 Αρχιτεκτονική Συστήματος 11
1.1 Επισκόπηση 11
1.2 Τεχνολογικές Επιλογές 11
1.3 Δομή Φακέλων Έργου 12
2 Σχεδίαση Βάσης Δεδομένων 14
2.1 Οντότητες και Σχέσεις (ER) 14
2.2 Αντιστοίχηση στο DTD της Εκφώνησης 14
2.3 Migrations με Alembic 15
3 Ρόλοι Χρηστών και Ροή Εγγραφής 16
3.1 Αυθεντικοποίηση και Εξουσιοδότηση (JWT & RBAC) 18
4 Backend – REST API 19
4.1 Οργάνωση Κώδικα 19
4.2 Πλήρης Κατάλογος Endpoints 19
4.3 Τεκμηρίωση API (OpenAPI/ Swagger UI) 20
5 Διαχείριση Εκδηλώσεων 23
5.1 Δημιουργία Δημοσίευση 23
5.2 Επεξεργασία, Ακύρωση, Διαγραφή 25
5.3 Διαχείριση Χωρητικότητας και Διαθεσιμότητας 25
6 Αναζήτηση, Πλοήγηση και Σελιδοποίηση 27
7 Κρατήσεις και Ταυτόχρονη Πρόσβαση 28
8 Χάρτης OpenStreetMap 31
9 Σύστημα Μηνυμάτων (Messaging) 32
10 Εξαγωγή Δεδομένων σε XML / JSON 33
11 Αλγόριθμος Συστάσεων (Biased Matrix Factorization) 36
11.1 Θεωρητικό Υπόβαθρο 36
11.2 Σήμα Αλληλεπίδρασης (Implicit Feedback) 38

11.3 Cold-Start (Χρήστης Χωρίς Ιστορικό Κρατήσεων) 38
11.4 Έλεγχος (Testing) του Αλγορίθμου 39
12 Frontend – Οθόνες και Ροές Χρήστη 41
13 Ασφάλεια Εφαρμογής 42
14 Εγκατάσταση και Εκτέλεση 43
14.1 Ανάπτυξη (development) χωρίς Docker 43
14.2 Εκτέλεση μέσω Docker Compose (HTTPS) 43
14.3 Demo Δεδομένα και Λογαριασμοί Επίδειξης 44
15 Παραδοχές και Σχεδιαστικές Αποφάσεις 45
16 Επίλογος 46

Πίνακας εικόνων

Figure 1. Διάγραμμα αρχιτεκτονικής 11
Figure 2. Δομή Φακέλων 13
Figure 3. Οντότητες και Σχέσεις 14
Figure 4. Pending registration approval 17
Figure 5. Administrator Dashboard 17
Figure 6. Approve or Reject User 18
Figure 7. Swagger Ui Endpoints 21
Figure 8. Endpoint Example 21
Figure 9. Successful request 22
Figure 10. Create Event Form 23
Figure 11. Organizer Events Dashboard 24
Figure 12. Administrator Events Dashboard 24
Figure 13. Edit Event dashboard 25
Figure 14. Events Filters 27
Figure 15. Booking Event 29
Figure 16. Confirm Booking 29
Figure 17. Organizer Bookings Review 30
Figure 18. OpenStreetMap on Event Details 31
Figure 19. Organizer Messages 32
Figure 20. Organizer Review Messages 32
Figure 21. Export Events Buttons 33
Figure 22. XML exported 34
Figure 23. JSON exported 35
Figure 24. Events Recommendations 39
Figure 25. Frontend Views 41

Πίνακας πινάκων

Table 1. Τεχνολογικές επιλογές 11
Table 2. DTD 14
Table 3. Ρόλοι χρηστών 16
Table 4. Endpoints Catalog 19
Table 5. Events Search Endpoint 27
Table 6. Λογαριασμοί Επίδειξης 44
Table 7. Pending Λογαριασμοί 44

Ορισμοί, Ακρωνύμια και Συντομογραφίες

Συντομογραφία	Ορισμός
API	Application Programming Interface.
REST API	Representational State Transfer API; a web interface that uses HTTP methods such as GET, POST, PATCH, and DELETE to access resources.
HTTP	HyperText Transfer Protocol; a protocol for transferring data between a web browser and a web server.
HTTPS	HyperText Transfer Protocol Secure; the secure version of HTTP, encrypted using TLS.
SSL	Secure Sockets Layer; an older protocol for encrypting internet communications, largely replaced by TLS.
TLS	Transport Layer Security; a protocol for encrypting communication between a client and a server, used by the application through HTTPS.
JWT	JSON Web Token; a signed token that carries identity information and is used for stateless authentication.
RBAC	Role-Based Access Control; access control based on a user's role, such as admin, organizer, or participant.
JSON	JavaScript Object Notation; a lightweight format for exchanging structured data.
XML	eXtensible Markup Language; a format for representing hierarchical and structured data using tags.
DTD	Document Type Definition; a definition that specifies the permitted structure, order, and elements of an XML document.
HTML	HyperText Markup Language; a markup language for structuring web pages.
CSS	Cascading Style Sheets; a language for styling and presenting HTML and React elements.
SPA	Single Page Application; an application that loads one main page and dynamically updates its content without a full reload.
UI	User Interface; the interface through which the user interacts with the application.
UX	User Experience; the overall experience and usability of the application.
DOM	Document Object Model; a tree representation of an HTML page that can be updated by JavaScript.
URL	Uniform Resource Locator; the address of a resource on the internet.
URI	Uniform Resource Identifier; an identifier for a resource, such as an API path.
CRUD	Create, Read, Update, Delete; the four basic data-management operations.
ORM	Object-Relational Mapping; a technique for mapping programming objects to relational database tables.
SQL	Structured Query Language; a query language for relational databases.
DB	Database; the application uses PostgreSQL as its database.
RDBMS	Relational Database Management System; a system for managing relational databases.
PK	Primary Key; a key that uniquely identifies a record.
FK	Foreign Key; a key that links a record to a record in another table.
UUID	Universally Unique Identifier; a unique identifier used for users, events, bookings, and other entities.

Συντομογραφία	Ορισμός
ER	Entity-Relationship; a model of the entities and relationships in a database.
ERD	Entity-Relationship Diagram; a diagram that shows entities, attributes, and relationships.
PostgreSQL	An open-source relational database management system.
SQLite	A lightweight embedded relational database used by default for development without Docker.
Alembic	A migration tool for SQLAlchemy that manages the evolution of the database schema.
Migration	A controlled change to the database schema, such as adding a table or column.
Pydantic	A data-validation and serialization library used in FastAPI schemas.
FastAPI	A Python web framework for implementing REST APIs and automatically generating OpenAPI documentation.
OpenAPI	A standard for describing REST APIs on which Swagger UI and ReDoc are based.
Swagger UI	An interactive web page for exploring and testing API endpoints.
ReDoc	An alternative OpenAPI documentation format focused on readability.
CORS	Cross-Origin Resource Sharing; a mechanism that controls which origins are allowed to send requests to the API.
Origin	A combination of protocol, hostname, and port, such as https://localhost or http://localhost:5173.
SPA	Single Page Application; a frontend architecture in which React dynamically updates the UI.
Vite	A development and build tool for JavaScript and TypeScript frontend applications.
React	A JavaScript library for creating component-based user interfaces.
TypeScript	A language based on JavaScript with a static type system.
Node.js	A runtime environment for executing JavaScript outside the browser, used for the frontend build.
npm	Node Package Manager; a tool for installing JavaScript and TypeScript dependencies.
Docker	A containerization platform for packaging and running applications in isolated containers.
Docker Compose	A tool for defining and running multiple Docker services, such as PostgreSQL, the backend, and the web proxy.
Container	An isolated runtime environment in which an application service runs.
Reverse Proxy	A server that receives client requests and forwards them to the appropriate backend service.
Caddy	A web server and reverse proxy used for serving the static frontend and TLS termination.
TLS termination	The process in which the reverse proxy decrypts HTTPS traffic before forwarding the request to the backend.
Self-signed certificate	A certificate signed by the server itself rather than by a public Certificate Authority; used in the local demonstration.
CA	Certificate Authority; a trusted authority that issues and digitally signs certificates.
OpenStreetMap	An open map used to display the geographic location of an event.
API endpoint	A specific path and HTTP method through which a REST API function is provided.
HTTP method	An HTTP method that specifies the action of a request, such as GET, POST, PATCH, or DELETE.
Middleware	Intermediate software that processes requests and responses before or after an endpoint.
Authentication	The process of verifying a user's identity.
Authorization	The process of checking whether an authenticated user has permission to perform an action.

Συντομογραφία	Ορισμός
Access token	A JWT token sent by the client to the API to access protected resources.
Password hashing	A one-way transformation of a password into a hash; the application uses Argon2id.
Argon2id	A modern password-hashing algorithm designed to resist brute-force and GPU attacks.
Seed / Seeding	The process of initially inserting data into the database, such as an admin account, demo events, or dataset records.
Idempotent seed	A seed process that can be run multiple times without creating duplicate records.
CSV	Comma-Separated Values; a simple format for storing data in rows and columns.
Dataset	A collection of data used for analysis, testing, or algorithm training.
EventView	A record that a signed-in user viewed an event; used as implicit feedback.
Implicit feedback	An indirect indication of user interest, such as a view or booking, rather than an explicit rating.
Booking	A record of a ticket or seat reservation for a specific event and ticket type.
Overbooking	Accepting more ticket reservations than the number of available seats.
Concurrency	The simultaneous execution of multiple requests, such as several users booking the final available seats.
Atomic update	An update executed as one indivisible database operation to prevent race conditions.
Transaction	A group of database operations that either all complete together or all fail together.
Race condition	An error that occurs when the result depends on the timing or order of concurrent actions.
SGD	Stochastic Gradient Descent; an iterative optimization algorithm used to train the recommender system.
MF	Matrix Factorization; a technique that factorizes an interaction matrix into latent vectors.
Biased Matrix Factorization	A variation of Matrix Factorization that includes global, user, and item biases.
Latent factor	A hidden numerical dimension that captures an underlying user or event characteristic.
Global bias	The average of all interaction scores in the recommender system.
User bias	A user's individual deviation from the global average.
Item bias	An event's deviation from the global average.
Regularization	A technique used to limit overfitting during model training.
Cold-start	A case in which a user or event has little or no interaction history.
Fallback	Alternative behavior used when there is not enough data for a personalized recommendation.
Demo data	Predefined demonstration data used for testing the application.
Admin	Application Administrator; the administrator of the application.
UI	User Interface; the graphical user interface.
API docs	Documentation of API endpoints, schemas, parameters, and responses.
HTTP 401	Unauthorized; authentication failed or a valid token is missing.
HTTP 403	Forbidden; the user is authenticated but does not have sufficient permissions.
HTTP 404	Not Found; the requested resource could not be found.
HTTP 409	Conflict; a state conflict occurred, such as insufficient ticket availability.
HTTP 422	Unprocessable Entity; the request data does not satisfy the validation rules.
HTTP 500	Internal Server Error; an unexpected error occurred on the server.

1 Αρχιτεκτονική Συστήματος

1.1 Επισκόπηση

Στιγμιότυπο του διαγράμματος αρχιτεκτονικής:

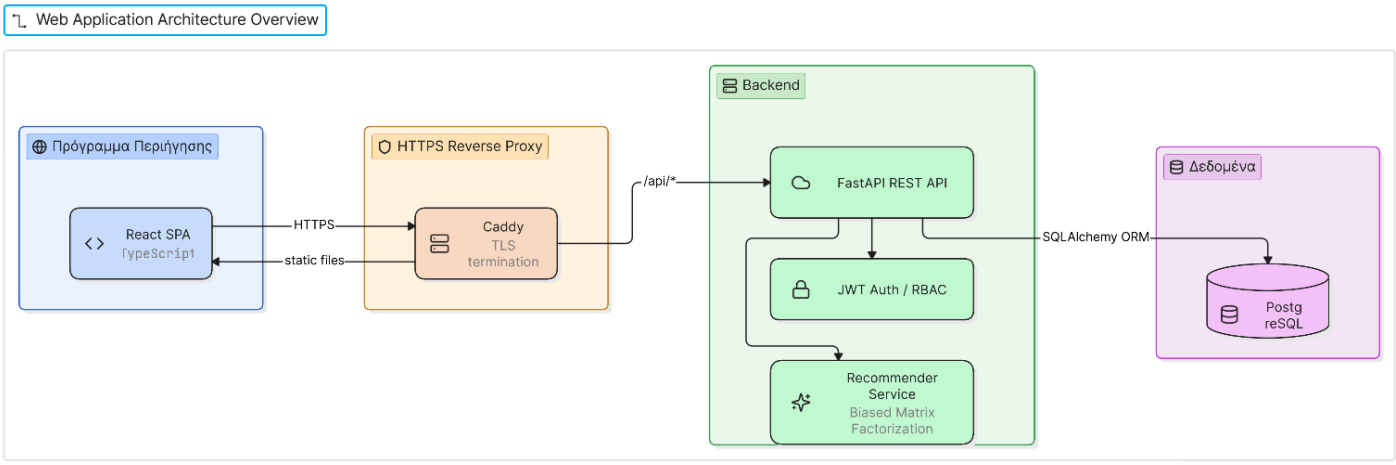


Figure 1. Διάγραμμα αρχιτεκτονικής

Η ροή μιας τυπικής αίτησης είναι η εξής:

1. Ο χρήστης αλληλεπιδρά με το React SPA μέσα από τον browser.
2. Το SPA καλεί το REST API μέσω `fetch`, χρησιμοποιώντας JWT στο header `Authorization: Bearer <token>` όπου απαιτείται.
3. Το FastAPI εξυπηρετεί το αίτημα, ελέγχει την ταυτότητα/εξουσιοδότηση, εκτελεί την επιχειρησιακή λογική και επικοινωνεί με την PostgreSQL μέσω SQLAlchemy.
4. Η απάντηση επιστρέφει ως JSON.
5. Σε παραγωγικό περιβάλλον (`docker compose up`), ο Caddy αναλαμβάνει τον τερματισμό TLS και προωθεί (`reverse_proxy`) τα αιτήματα `/api/*` στο backend, ενώ σερβίρει το στατικό build του frontend — έτσι όλη η επικοινωνία με τον client γίνεται πάνω από HTTPS, όπως απαιτεί η εκφώνηση.

1.2 Τεχνολογικές Επιλογές

Table 1. Τεχνολογικές επιλογές

Επίπεδο	Τεχνολογία	Αιτιολόγηση
Backend framework	FastAPI	Αυτόματη τεκμηρίωση OpenAPI, ενσωματωμένη υποστήριξη Pydantic για επικύρωση δεδομένων, ασύγχρονη υποστήριξη.

Επίπεδο	Τεχνολογία	Αιτιολόγηση
ORM	SQLAlchemy	Ώριμο, πλήρες mapping σχεσιακού → αντικειμενοστρεφούς μοντέλου, όπως ζητά η εκφώνηση.
Migrations	Alembic	Ελεγχόμενη εξέλιξη σχήματος βάσης δεδομένων, autogenerate από τα μοντέλα.
Βάση Δεδομένων	PostgreSQL 16	Πλήρης υποστήριξη συναλλαγών (transactions) και κλειδωμάτων γραμμών, απαραίτητη για την αποφυγή υπερκράτησης θέσεων (overbooking).
Frontend framework	React 18 + TypeScript	Απαιτείται ρητά JavaScript/TypeScript framework. Το TypeScript προσφέρει στατικό έλεγχο τύπων.
Build tool	Vite	Ταχύτατο dev server και βελτιστοποιημένο production build.
Πιστοποίηση	JWT (PyJWT)	Ασφαλές hashing κωδικών και stateless αυθεντικοποίηση.
Reverse proxy / TLS	Caddy	Απλή διαμόρφωση HTTPS με αυτόματη διαχείριση πιστοποιητικών.
Containerization	Docker	Αναπαραγώγιμη εγκατάσταση `postgres` + `backend` + `web`.

1.3 Δομή Φακέλων Έργου

project/

- **backend/**
  - **app/**
    - main.py — Είσοδος εφαρμογής, CORS, seed διαχειριστή
    - config.py — Ρυθμίσεις μέσω μεταβλητών περιβάλλοντος
    - db.py — SQLAlchemy engine / session
    - models.py — Οντότητες ORM (User, Event, Booking, ...)
    - schemas.py — Pydantic μοντέλα εισόδου/εξόδου
    - security.py — JWT, hashing, εξαρτήσεις ρόλων
    - **routers/** — auth, events, bookings, messages, admin, recommendations
    - **services/**
      - recommendations.py — Biased Matrix Factorization από το μηδέν
  - **alembic/** — Migrations βάσης δεδομένων
  - **tests/** — 26 αυτοματοποιημένα tests (pytest)

    - Dockerfile
    - pyproject.toml
- **frontend/**
    - **src/**
        - App.tsx — Όλες οι οθόνες/ροές του SPA
        - main.tsx
        - styles.css
    - Dockerfile — Multi-stage build → Caddy image
    - Caddyfile — Διαμόρφωση HTTPS reverse proxy
    - package.json
- docker-compose.yml — postgres + backend + web (HTTPS)
- README.md

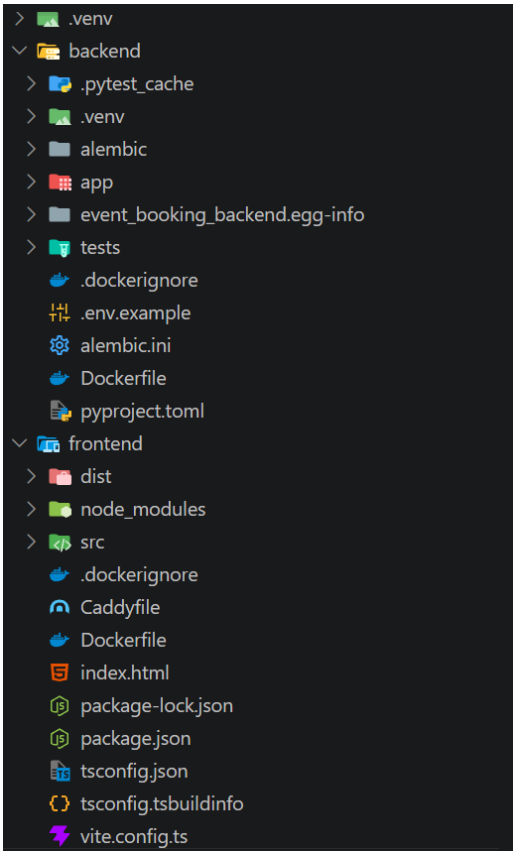


Figure 2. Δομή Φακέλων

2 Σχεδίαση Βάσης Δεδομένων

2.1 Οντότητες και Σχέσεις (ER)

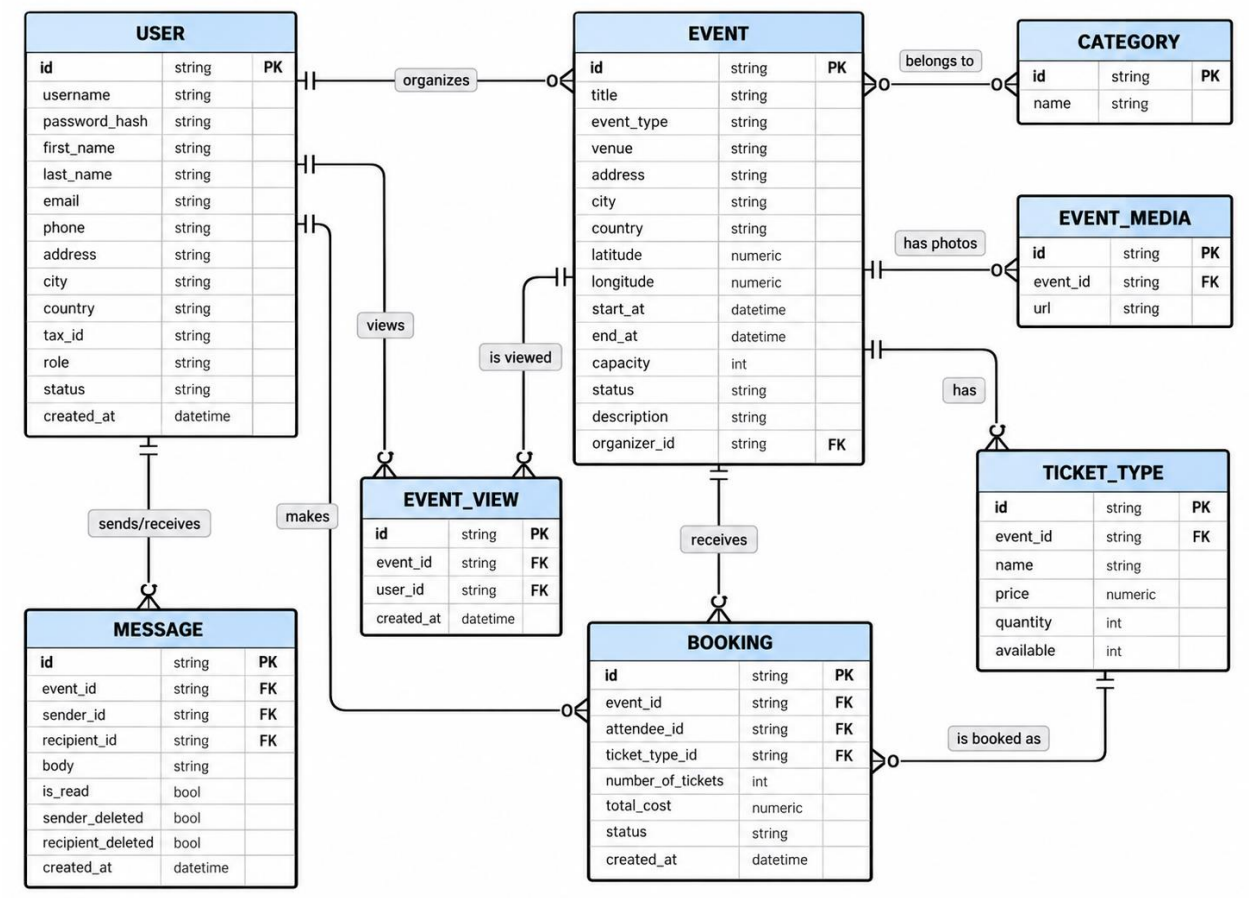


Figure 3. Οντότητες και Σχέσεις

2.2 Αντιστοίχηση στο DTD της Εκφώνησης

Το σχεσιακό μοντέλο σχεδιάστηκε ώστε να απεικονίζει πιστά το DTD που δόθηκε στην εκφώνηση:

Table 2. DTD

Στοιχείο DTD	Αναπαράσταση στη Βάση Δεδομένων
`Event/@EventID`	`events.id` (UUID)
`Title`, `EventType`, `Venue`, `Address`, `City`, `Country`, `Description`	Πεδία του πίνακα `events`

`Category+`	Πίνακας `categories` + πίνακας σύνδεσης `event_categories` (πολλά-προς-πολλά)
`GeoLocation` (προαιρετικό)	`events.latitude`, `events.longitude` (nullable)
`StartDateTime`, `EndDateTime`	`events.start_at`, `events.end_at`
`Capacity`	`events.capacity`
`TicketTypes/TicketType`	Πίνακας `ticket_types` (name, price, quantity, available)
`Bookings/Booking`	Πίνακας `bookings` (attendee, time, ticket type αναφορά, αριθμός εισιτηρίων, συνολικό κόστος, κατάσταση)
`Organizer`	`events.organizer_id` (foreign key στο `users`)
`Status`	`events.status` (`DRAFT`, `PUBLISHED`, `COMPLETED`, `CANCELLED`)
`Media/Photo*` (προαιρετικό)	Πίνακας `event_media` (μία γραμμή ανά URL φωτογραφίας)

Η εξαγωγή XML παράγει ακριβώς αυτή τη δομή, με τη σειρά στοιχείων να ακολουθεί αυστηρά τη σειρά που ορίζει το DTD.

2.3 Migrations με Alembic

Το σχήμα της βάσης δεδομένων δεν δημιουργείται χειροκίνητα, αλλά μέσω ελεγχόμενων migrations:

- `9f2815a8d5f9_initial_schema.py` — αρχικό σχήμα (χρήστες, εκδηλώσεις, κατηγορίες, τύποι εισιτηρίων, κρατήσεις, μηνύματα, προβολές εκδηλώσεων).
- `c86062cd520f_add_event_media.py` — προσθήκη του πίνακα `event_media` για την υποστήριξη προαιρετικών φωτογραφιών.

Κάθε migration παράχθηκε αυτόματα (`alembic revision --autogenerate`) με βάση τα μοντέλα SQLAlchemy και επαληθεύτηκε έμπρακτα με πραγματική εκτέλεση `upgrade` → `downgrade` → `upgrade` σε running instance PostgreSQL, ώστε να διασφαλιστεί ότι είναι αντιστρέψιμα και ορθά.

3 Ρόλοι Χρηστών και Ροή Εγγραφής

Η εφαρμογή υποστηρίζει τους τέσσερις ρόλους της εκφώνησης:

Table 3. Ρόλοι χρηστών

Ρόλος	Δυνατότητες
Επισκέπτης (μη συνδεδεμένος)	Πλοήγηση/αναζήτηση εκδηλώσεων, προβολή λεπτομερειών χωρίς δυνατότητα κράτησης.
Συμμετέχων	Αναζήτηση, προβολή, κράτηση θέσεων, μηνύματα με διοργανωτή, προβολή συστάσεων.
Διοργανωτής	Δημιουργία/επεξεργασία/δημοσίευση/ακύρωση/διαγραφή εκδηλώσεων, προβολή κρατήσεων ανά εκδήλωση, μηνύματα με συμμετέχοντες.
Διαχειριστής	Έγκριση/απόρριψη εγγραφών, προβολή στοιχείων χρηστών, εξαγωγή δεδομένων σε XML/JSON.

- Ο χρήστης πατά «Create account» στη σελίδα καλωσορίσματος.
- Συμπληρώνει: όνομα χρήστη, κωδικό, επιβεβαίωση κωδικού, όνομα, επώνυμο, email, τηλέφωνο, διεύθυνση, πόλη, χώρα, ΑΦΜ και επιλέγει επιθυμητό ρόλο (Συμμετέχων ή Διοργανωτής).
- Το backend ελέγχει τη μοναδικότητα του username/email και την ταύτιση κωδικού/επιβεβαίωσης (βλ. `UserCreate` schema).
- Ο λογαριασμός δημιουργείται με κατάσταση `PENDING`.
- Ο χρήστης ενημερώνεται ότι εκκρεμεί έγκριση από τον διαχειριστή.
- Ο διαχειριστής (ενσωματωμένος λογαριασμός `admin` κατά την εγκατάσταση) βλέπει τη λίστα εκκρεμών χρηστών, ανοίγει τη σελίδα λεπτομερειών κάθε χρήστη και εγκρίνει ή απορρίπτει την αίτηση.
- Μόνο μετά την έγκριση (`status = APPROVED`) επιτρέπεται η είσοδος (login) του χρήστη.

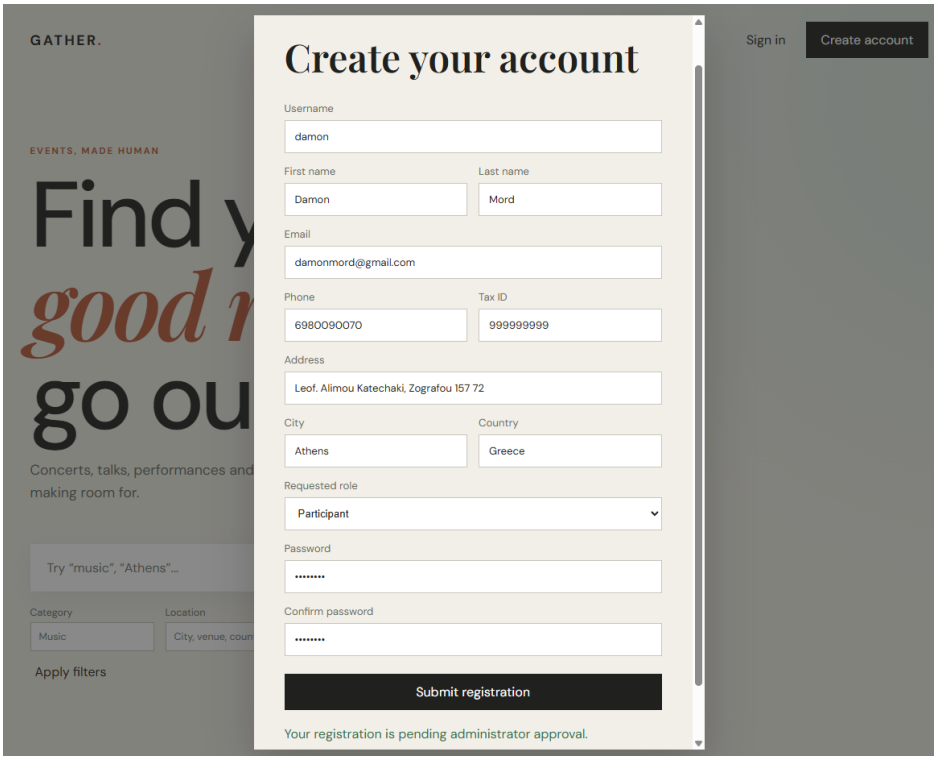


Figure 4. Pending registration approval

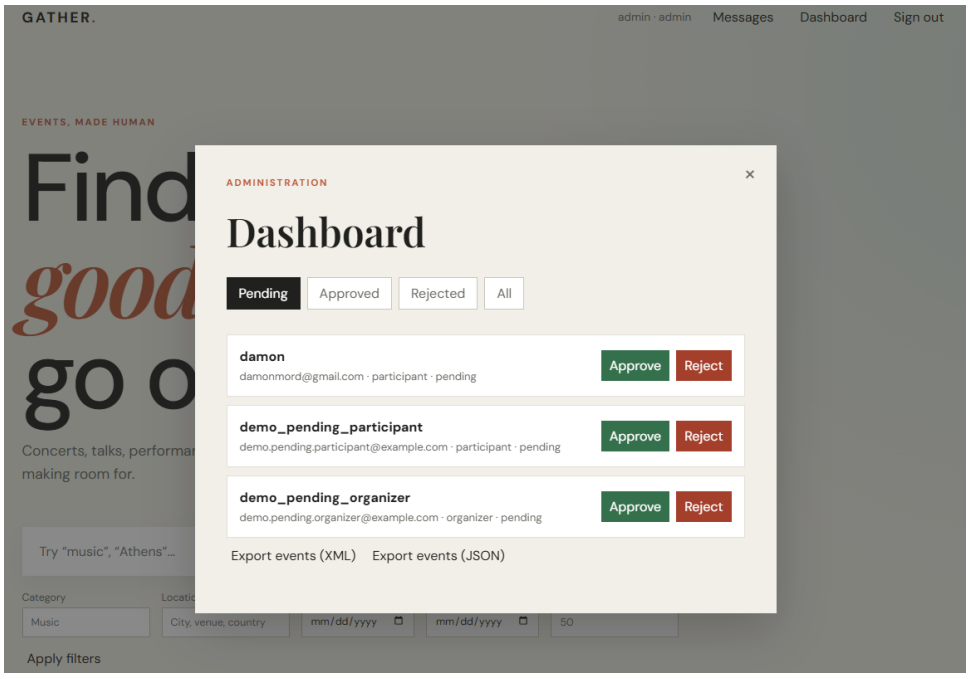


Figure 5. Administrator Dashboard

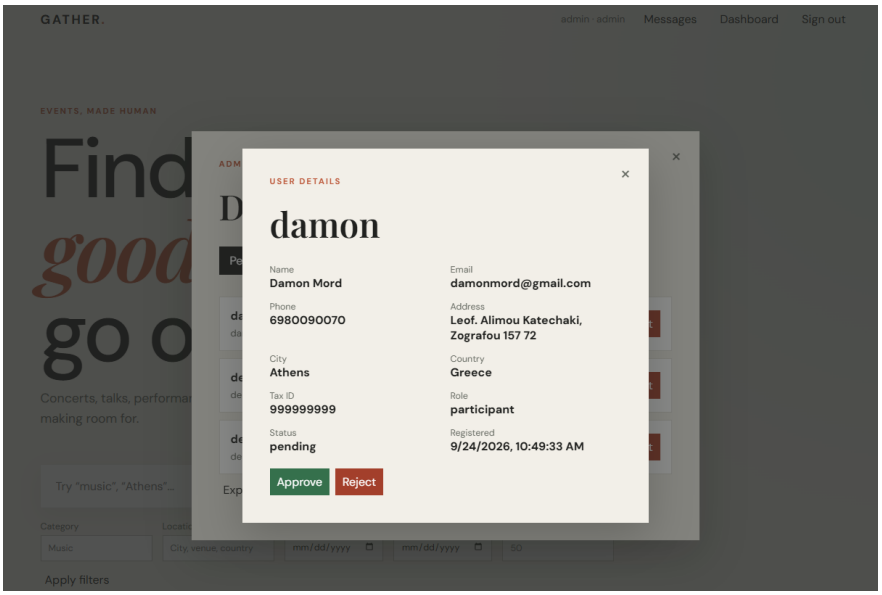


Figure 6. Approve or Reject User

3.1 Αυθεντικοποίηση και Εξουσιοδότηση (JWT & RBAC)

- Οι κωδικοί πρόσβασης αποθηκεύονται πάντα ως **hash Argon2id** (βιβλιοθήκη `pwdlib`), ποτέ σε καθαρό κείμενο.
- Κατά το login (`POST /api/auth/login`), αν τα στοιχεία είναι σωστά **και** ο λογαριασμός έχει εγκριθεί, εκδίδεται **JWT access token** (αλγόριθμος HS256) με:
- `sub`: το αναγνωριστικό του χρήστη,
- `role`: τον ρόλο του χρήστη,
- `exp`: χρόνο λήξης (30 λεπτά, ρυθμιζόμενο).
- Κάθε προστατευμένο endpoint απαιτεί το header `Authorization: Bearer <token>`.
- Η εξουσιοδότηση υλοποιείται μέσω FastAPI dependency injection:
- `get_current_user`: αποκωδικοποιεί/επικυρώνει το JWT και επιστρέφει τον χρήστη (απορρίπτει ανενεργούς ή μη εγκεκριμένους λογαριασμούς).
- `require_roles(*roles)`: επιστρέφει HTTP 403 αν ο ρόλος του χρήστη δεν περιλαμβάνεται στους επιτρεπόμενους.
- Τα endpoints δημόσιας αναζήτησης/προβολής εκδηλώσεων (`GET /api/events`, `GET /api/events/{id}`) **δεν** απαιτούν JWT, ώστε να ικανοποιείται η απαίτηση πρόσβασης επισκέπτη χωρίς σύνδεση. Όλες οι υπόλοιπες ενέργειες (κράτηση, μηνύματα, διαχείριση, εξαγωγές) απαιτούν έγκυρο JWT και τον κατάλληλο ρόλο.

4 Backend – REST API

4.1 Οργάνωση Κώδικα

Το backend είναι οργανωμένο σε ξεχωριστά modules ανά αρμοδιότητα (routers), σύμφωνα με τις αρχές διαχωρισμού ευθυνών:

- `routers/auth.py` — εγγραφή, login, `/me`.
- `routers/admin.py` — διαχείριση χρηστών, εξαγωγές XML/JSON.
- `routers/events.py` — CRUD εκδηλώσεων, αναζήτηση/σελιδοποίηση, κύκλος ζωής, κρατήσεις ανά εκδήλωση.
- `routers/bookings.py` — δημιουργία κράτησης, ιστορικό κρατήσεων χρήστη.
- `routers/messages.py` — εισερχόμενα/απεσταλμένα, μη αναγνωσμένα, διαγραφή.
- `routers/recommendations.py` — καταγραφή προβολών, σύσταση εκδηλώσεων.
- `services/recommendations.py` — καθαρή υλοποίηση του αλγορίθμου Biased Matrix Factorization, αποσυζευγμένη από το HTTP layer.

4.2 Πλήρης Κατάλογος Endpoints

Table 4. Endpoints Catalog

Μέθοδος	Διαδρομή	Ρόλος	Περιγραφή
POST	`/api/auth/register`	Δημόσιο	Εγγραφή νέου χρήστη (κατάσταση `PENDING`)
POST	`/api/auth/login`	Δημόσιο	Είσοδος, έκδοση JWT
GET	`/api/auth/me`	Συνδεδεμένος	Στοιχεία τρέχοντος χρήστη
GET	`/api/admin/users`	Admin	Λίστα χρηστών (φίλτρο κατάστασης)
GET	`/api/admin/users/{id}`	Admin	Λεπτομέρειες συγκεκριμένου χρήστη
POST	`/api/admin/users/{id}/approve`	Admin	Έγκριση εγγραφής
POST	`/api/admin/users/{id}/reject`	Admin	Απόρριψη εγγραφής
GET	`/api/admin/exports/events.json`	Admin	Εξαγωγή όλων των εκδηλώσεων σε JSON
GET	`/api/admin/exports/events.xml`	Admin	Εξαγωγή όλων των εκδηλώσεων σε XML (κατά DTD)
GET	`/api/events`	Δημόσιο	Αναζήτηση/πλοήγηση με φίλτρα + σελιδοποίηση
GET	`/api/events/{id}`	Δημόσιο	Λεπτομέρειες δημοσιευμένης εκδήλωσης
POST	`/api/events`	Organizer	Δημιουργία νέας εκδήλωσης (κατάσταση `DRAFT`)
PATCH	`/api/events/{id}`	Organizer (ιδιοκτήτης)	Επεξεργασία στοιχείων εκδήλωσης

POST	`/api/events/{id}/publish`	Organizer (ιδιοκτήτης)	Δημοσίευση
POST	`/api/events/{id}/cancel`	Organizer (ιδιοκτήτης)	Ακύρωση + αυτόματη ειδοποίηση συμμετεχόντων
DELETE	`/api/events/{id}`	Organizer (ιδιοκτήτης)	Διαγραφή (υπό όρους)
GET	`/api/events/organizer/mine`	Organizer	Οι εκδηλώσεις του συνδεδεμένου διοργανωτή
GET	`/api/events/{id}/bookings`	Organizer (ιδιοκτήτης)	Κρατήσεις μιας συγκεκριμένης εκδήλωσης
POST	`/api/events/{id}/bookings`	Participant	Υποβολή κράτησης
GET	`/api/bookings`	Participant	Ιστορικό κρατήσεων συνδεδεμένου χρήστη
POST	`/api/events/{id}/views`	Συνδεδεμένος	Καταγραφή προβολής εκδήλωσης (για συστάσεις)
GET	`/api/recommendations`	Participant	Προσωποποιημένες συστάσεις
POST	`/api/messages`	Συνδεδεμένος	Αποστολή μηνύματος (μετά από κράτηση)
GET	`/api/messages/inbox`	Συνδεδεμένος	Εισερχόμενα
GET	`/api/messages/sent`	Συνδεδεμένος	Απεσταλμένα
GET	`/api/messages/unread-count`	Συνδεδεμένος	Πλήθος μη αναγνωσμένων
DELETE	`/api/messages/{id}`	Συνδεδεμένος	Διαγραφή από το προσωπικό mailbox
GET	`/api/health`	Δημόσιο	Έλεγχος διαθεσιμότητας υπηρεσίας

4.3 Τεκμηρίωση API (OpenAPI/ Swagger UI)

Το FastAPI παράγει αυτόματα διαδραστική τεκμηρίωση στο `/docs` (Swagger UI) και `/redoc`, βάσει των Pydantic schemas εισόδου/εξόδου.

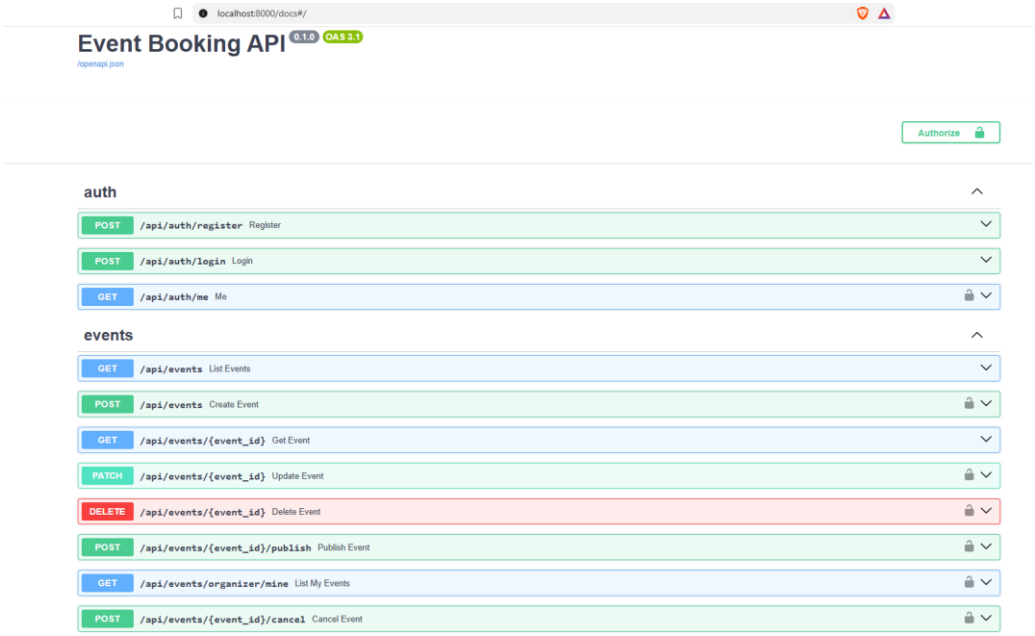


Figure 7. Swagger Ui Endpoints

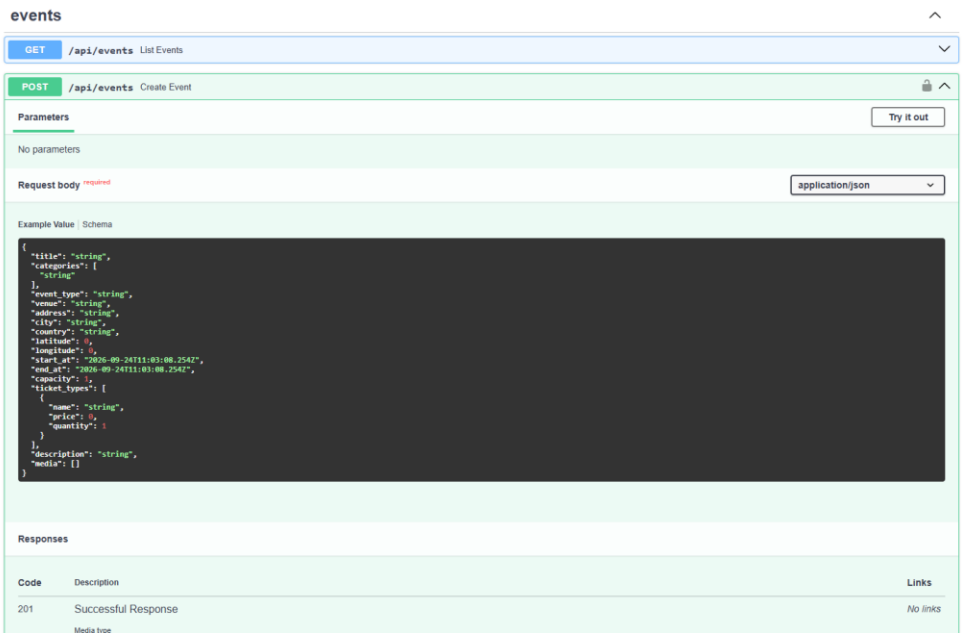


Figure 8. Endpoint Example

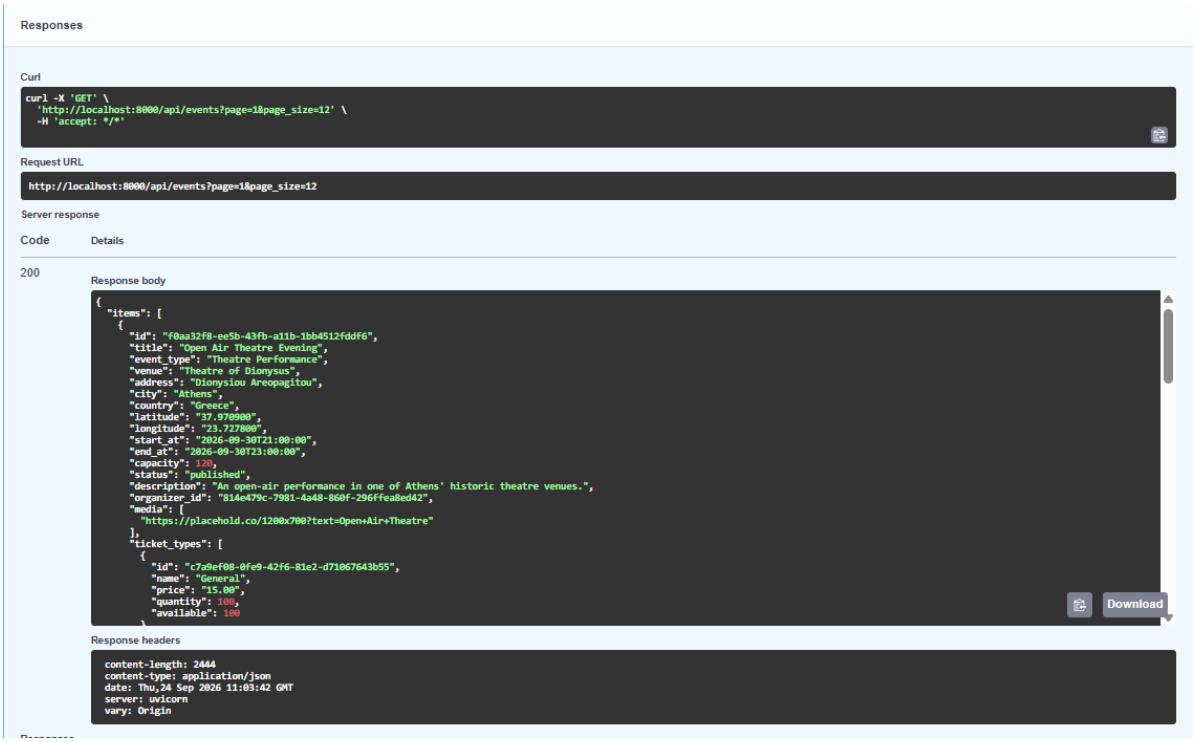


Figure 9. Successful request

5 Διαχείριση Εκδηλώσεων

5.1 Δημιουργία Δημοσίευση

Ο διοργανωτής δημιουργεί μια εκδήλωση μέσω φόρμας που περιλαμβάνει όλα τα πεδία του DTD: τίτλο, κατηγορίες (comma-separated στο UI, μετατρέπονται σε λίστα), τύπο εκδήλωσης, χώρο διεξαγωγής, διεύθυνση, πόλη, χώρα, προαιρετικές γεωγραφικές συντεταγμένες, ημερομηνίες έναρξης/λήξης, χωρητικότητα, τύπους εισιτηρίων (όνομα/τιμή/ποσότητα) και προαιρετικές φωτογραφίες (URLs). Η εκδήλωση δημιουργείται σε κατάσταση `DRAFT`. Ο διοργανωτής μπορεί στη συνέχεια να την δημοσιεύσει (`PUBLISHED`), οπότε γίνεται ορατή στην αναζήτηση.

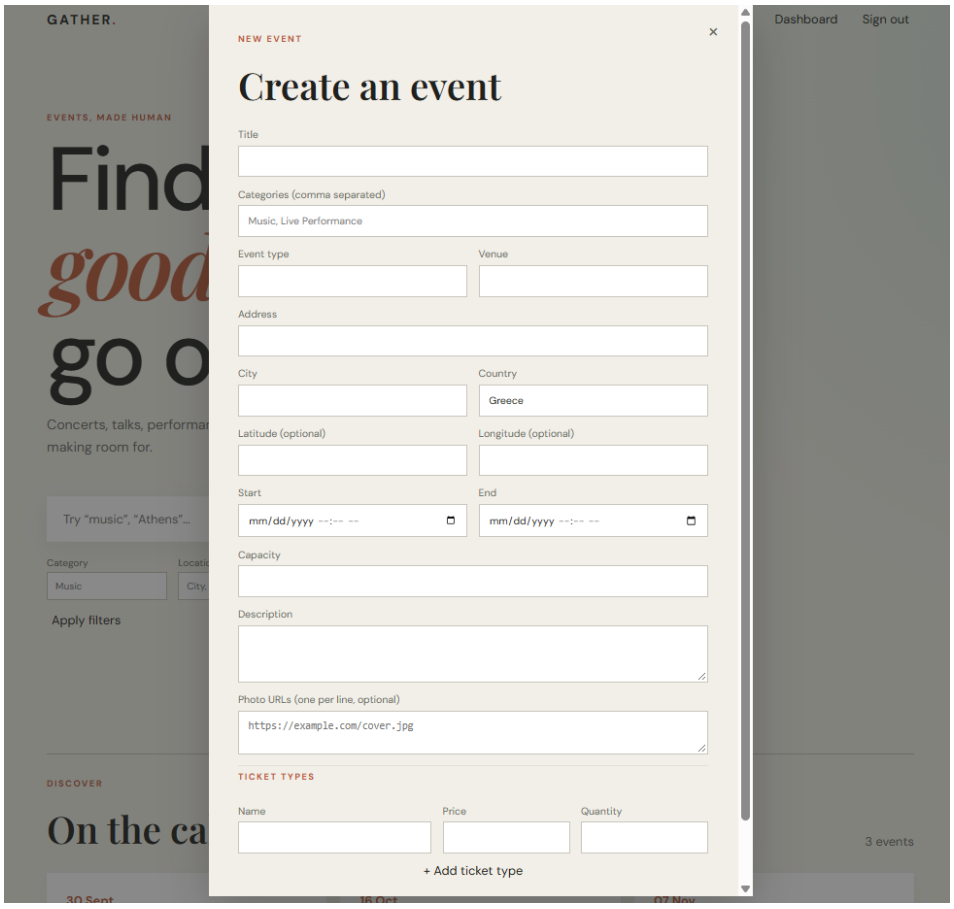


Figure 10. Create Event Form

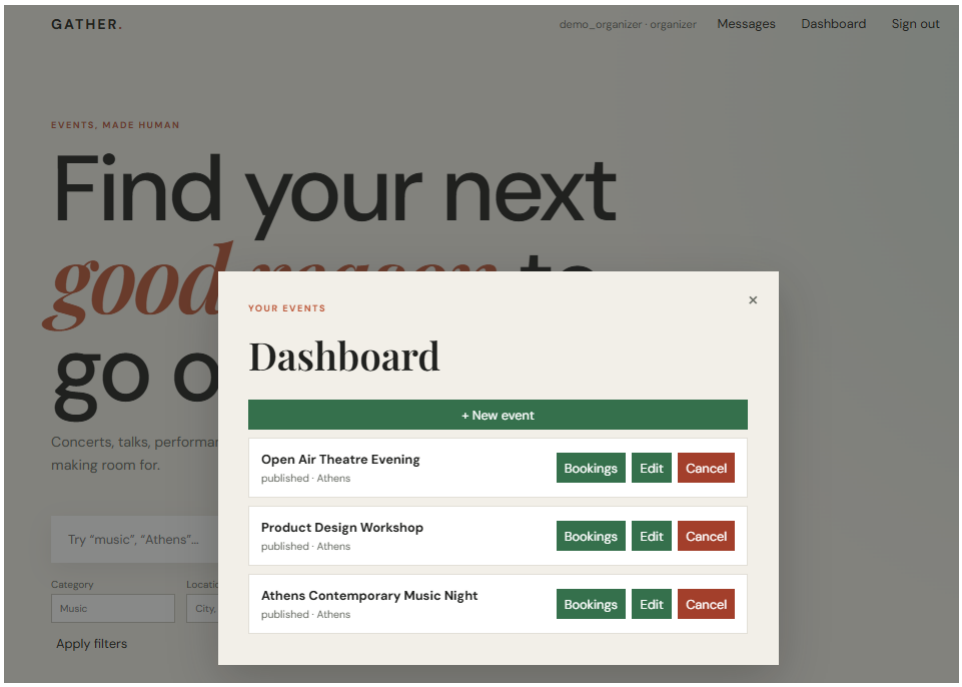


Figure 11. Organizer Events Dashboard

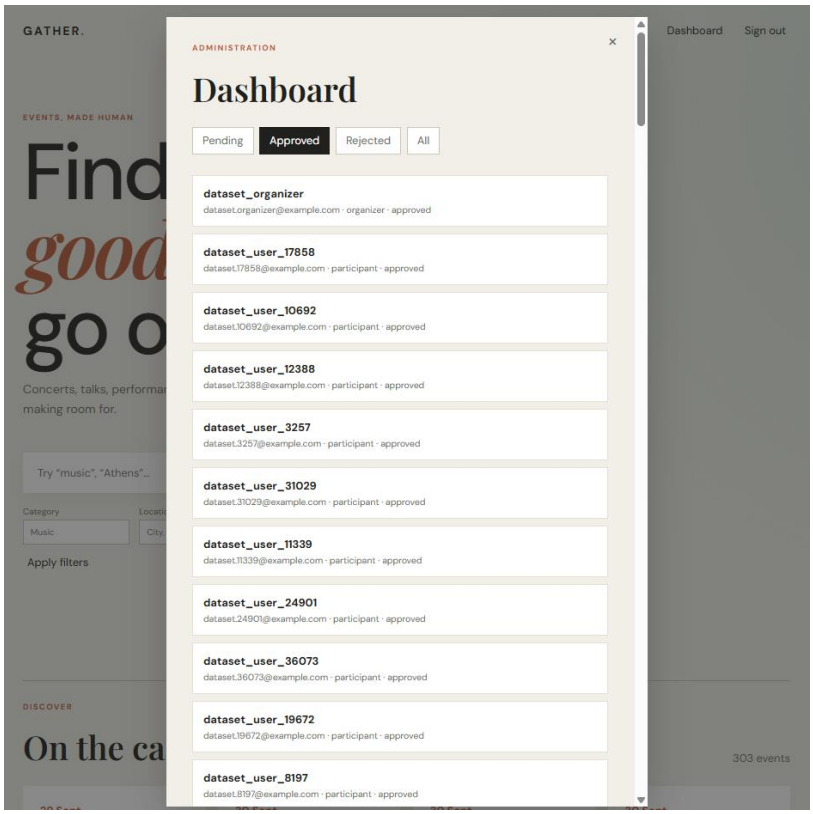


Figure 12. Administrator Events Dashboard

5.2 Επεξεργασία, Ακύρωση, Διαγραφή

Υλοποιήθηκαν ακριβώς οι κανόνες της εκφώνησης:

- **Διαγραφή** επιτρέπεται μόνο όσο η εκδήλωση είναι `DRAFT` **και** δεν υπάρχει καμία κράτηση.
- **Ακύρωση** επιτρέπεται μόνο για ήδη δημοσιευμένες (`PUBLISHED`) εκδηλώσεις. Η κατάσταση μεταβαίνει σε `CANCELLED`, χωρίς διαγραφή δεδομένων.
- Μετά την ακύρωση, **δεν επιτρέπονται νέες κρατήσεις** (ελέγχεται στο endpoint κράτησης).
- Οι υπάρχουσες κρατήσεις **παραμένουν αποθηκευμένες** για ιστορικότητα/ιχνηλασιμότητα.
- Κατά την ακύρωση, το σύστημα στέλνει αυτόματα ενημερωτικό μήνυμα σε **όλους** τους χρήστες με επιβεβαιωμένη κράτηση για την εκδήλωση.

Ολοκληρωμένες (`COMPLETED`) ή ακυρωμένες εκδηλώσεις δεν μπορούν πλέον να επεξεργαστούν

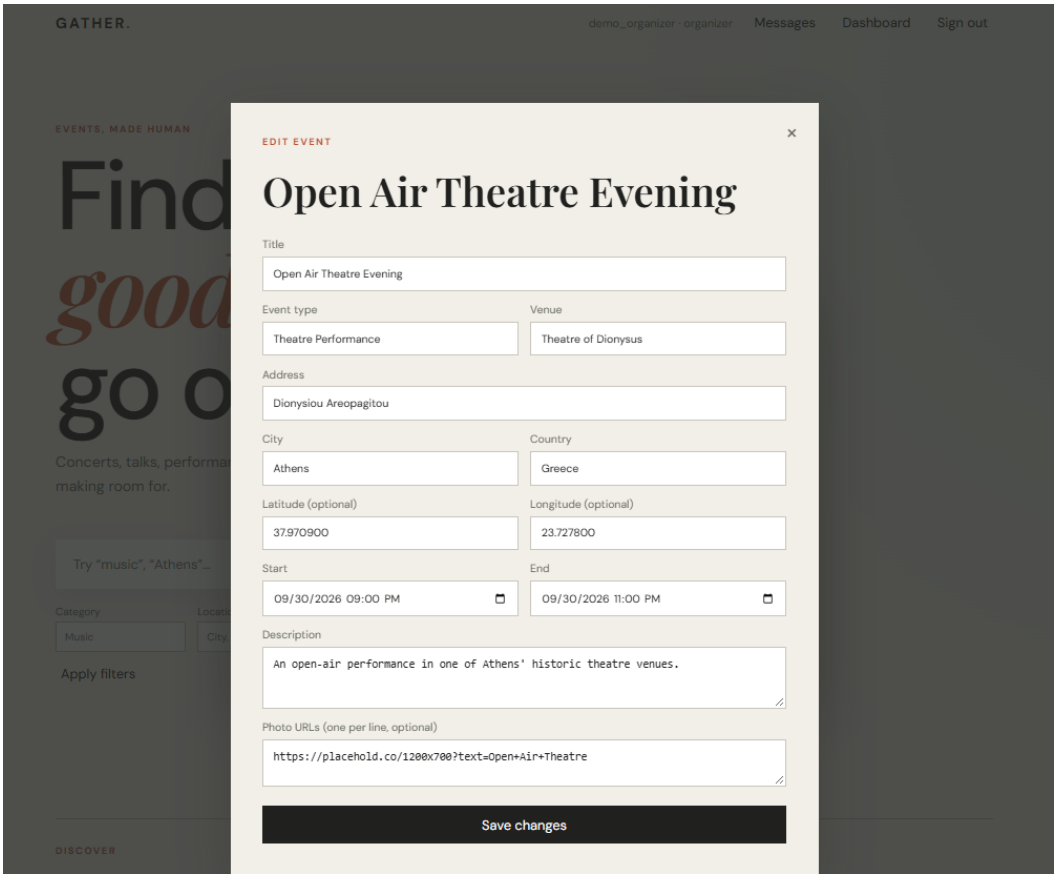


Figure 13. Edit Event dashboard

5.3 Διαχείριση Χωρητικότητας και Διαθεσιμότητας

Για κάθε εκδήλωση τηρούνται:

- η συνολική χωρητικότητα (`capacity`),

- ανά τύπο εισιτηρίου: η αρχική ποσότητα (`quantity`) και η τρέχουσα διαθεσιμότητα (`available`),
- έμμεσα, το σύνολο δεσμευμένων θέσεων (= `quantity − available`, αθροιστικά ανά τύπο εισιτηρίου).

Κατά τη δημιουργία ή επεξεργασία τύπων εισιτηρίων, το backend επικυρώνει ότι το άθροισμα των ποσοτήτων δεν ξεπερνά τη χωρητικότητα της εκδήλωσης (HTTP 422 διαφορετικά). Κατά την κράτηση, ελέγχεται ότι υπάρχει επαρκής διαθεσιμότητα πριν επιτραπεί η συναλλαγή.

6 Αναζήτηση, Πλοήγηση και Σελιδοποίηση

Το endpoint `GET /api/events` υποστηρίζει όλα τα κριτήρια που ζητά η εκφώνηση:

Table 5. Events Search Endpoint

Παράμετρος	Περιγραφή
`search`	Ελεύθερο κείμενο σε τίτλο και περιγραφή
`category`	Φιλτράρισμα βάσει κατηγορίας
`location`	Ελεύθερο κείμενο σε πόλη, χώρα, χώρο διεξαγωγής, διεύθυνση
`city`	Ακριβές φιλτράρισμα πόλης
`date_from` / `date_to`	Χρονικό διάστημα έναρξης εκδήλωσης
`min_price` / `max_price`	Εύρος τιμής εισιτηρίου
`page` / `page_size`	Σελιδοποίηση αποτελεσμάτων

Η απάντηση είναι ένα «envelope» (`EventListResponse`) με `items`, `total`, `page`, `page_size`, ώστε το frontend να μπορεί να υλοποιήσει σωστή σελιδοποίηση (κουμπιά «Προηγούμενο/Επόμενο» και ένδειξη τρέχουσας σελίδας).

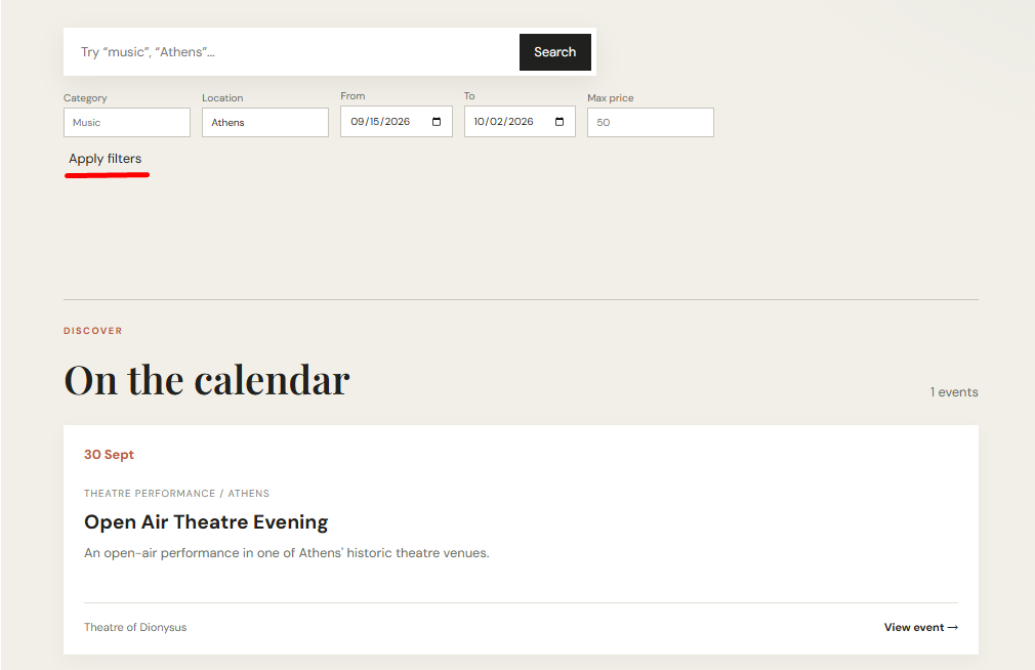


Figure 14. Events Filters

7 Κρατήσεις και Ταυτόχρονη Πρόσβαση

Η κράτηση θέσεων είναι το πιο κρίσιμο σημείο της εφαρμογής ως προς την ακεραιότητα δεδομένων, καθώς πολλοί χρήστες μπορεί να προσπαθήσουν να κρατήσουν τις τελευταίες διαθέσιμες θέσεις **ταυτόχρονα**.

Ροή κράτησης:

Ο συμμετέχων επιλέγει τύπο εισιτηρίου και αριθμό θέσεων.
Εμφανίζεται ρητή **οθόνη επιβεβαίωσης** («Confirm X × TicketType για €Y — δεν αναιρείται»), σύμφωνα με την απαίτηση της εκφώνησης ότι η κράτηση δεν μπορεί να αναιρεθεί μετά την οριστική υποβολή.
Μόνο μετά τη ρητή επιβεβαίωση αποστέλλεται το αίτημα `POST /api/events/{id}/bookings`.

Ασφάλεια ως προς τη συνέπεια δεδομένων (backend):

```
with db.begin_nested():
    result = db.execute(
        update(TicketType)
        .where(TicketType.id == ticket.id, TicketType.available >= payload.number_of_tickets)
        .values(available=TicketType.available - payload.number_of_tickets)
    )
    if result.rowcount != 1:
        raise HTTPException(status_code=409, detail="Not enough tickets available")
    booking = Booking(...)
    db.add(booking)
db.commit()
```

Η ενημέρωση της διαθεσιμότητας γίνεται με μια **ατομική (atomic) εντολή `UPDATE ... WHERE available >= N`** μέσα σε συναλλαγή. Αν δύο χρήστες προσπαθήσουν ταυτόχρονα να κλείσουν την τελευταία θέση, η βάση δεδομένων εγγυάται ότι μόνο η μία εντολή θα «πετύχει» (θα επηρεάσει γραμμή), ενώ η άλλη θα βρει `rowcount == 0` και θα απορριφθεί με HTTP 409, **χωρίς ποτέ να προκύψει υπερκράτηση (overbooking)**.

Αυτό επαληθεύτηκε πειραματικά με αυτοματοποιημένο test (`test_concurrent_bookings_never_oversell_tickets`) που εκτοξεύει **30 ταυτόχρονα αιτήματα κράτησης** (`ThreadPoolExecutor`) για εκδήλωση με χωρητικότητα 10 θέσεων· το αποτέλεσμα ήταν **ακριβώς 10 επιτυχείς κρατήσεις και 20 απορρίψεις**, με τελική διαθεσιμότητα 0 — καμία υπερκράτηση.

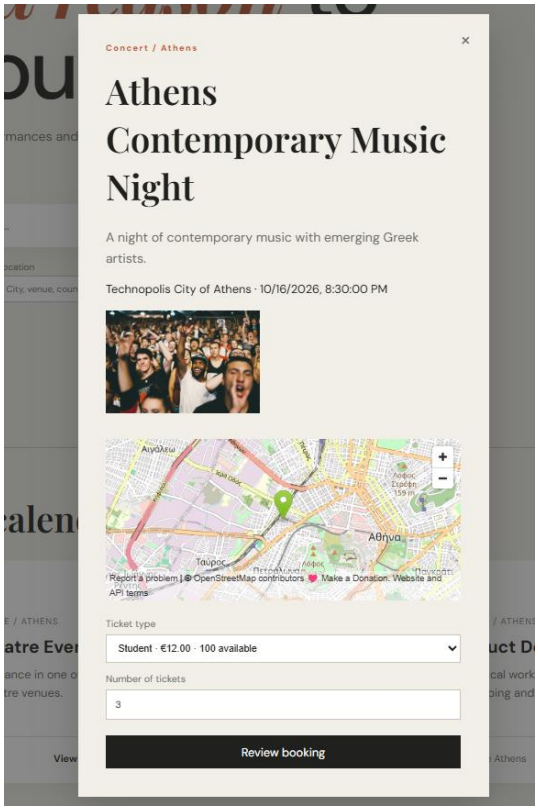


Figure 15. Booking Event

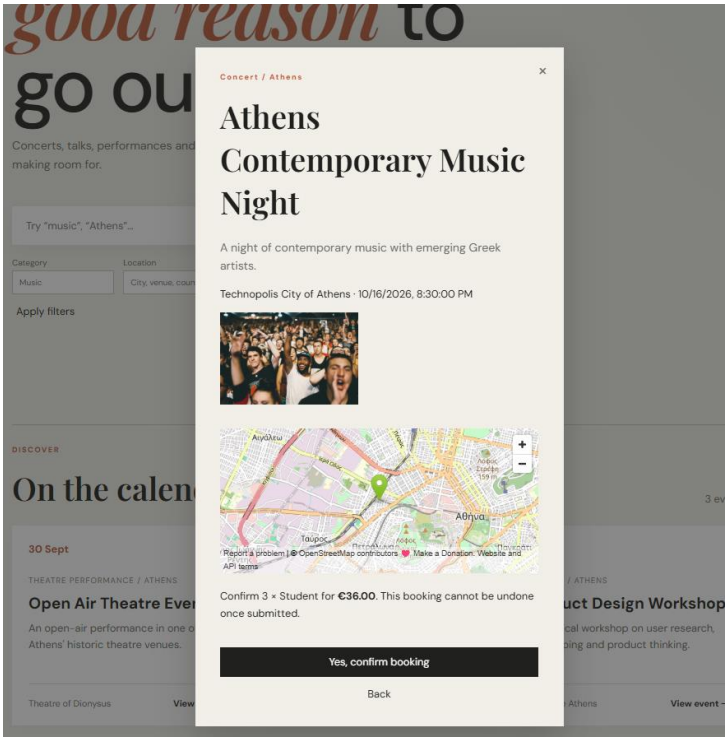


Figure 16. Confirm Booking

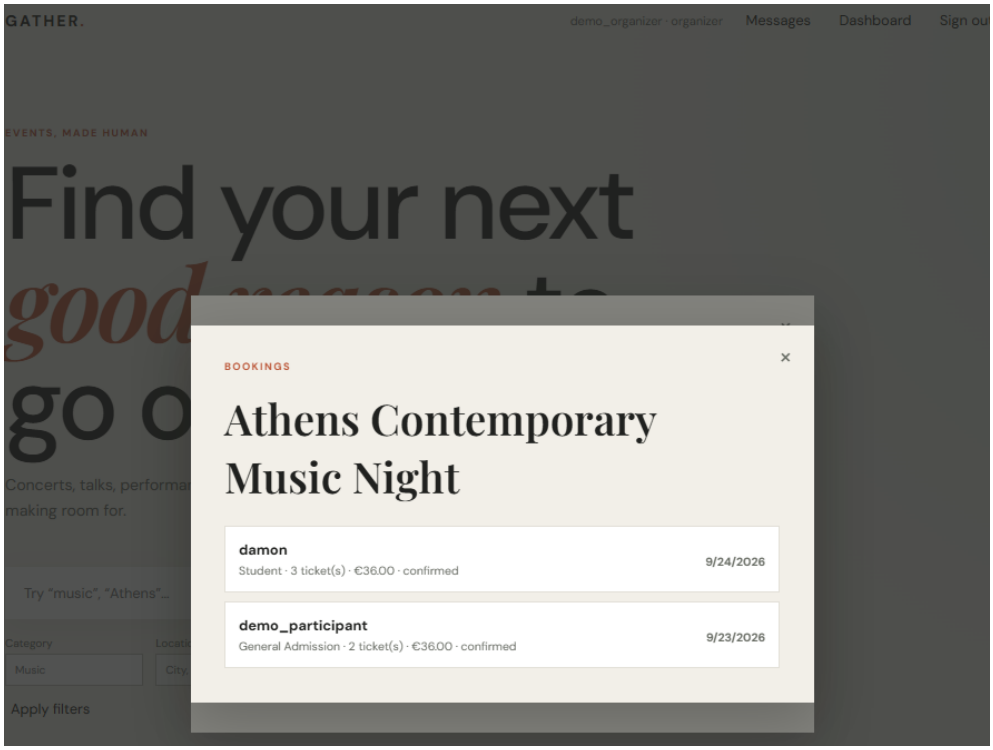


Figure 17. Organizer Bookings Review

8 Χάρτης OpenStreetMap

Στη σελίδα λεπτομερειών μιας εκδήλωσης, εφόσον έχουν οριστεί γεωγραφικές συντεταγμένες (`latitude`/`longitude`), εμφανίζεται ενσωματωμένος χάρτης **OpenStreetMap** μέσω `iframe` (`openstreetmap.org/export/embed.html`) με δείκτη (marker) στη θέση της εκδήλωσης, χωρίς να απαιτείται εξωτερικό API key.

Ο διοργανωτής εισάγει προαιρετικά το γεωγραφικό πλάτος και μήκος στη φόρμα δημιουργίας ή επεξεργασίας εκδήλωσης. Όταν ένας χρήστης ανοίξει τις λεπτομέρειες, το frontend δημιουργεί δυναμικά το URL του embedded χάρτη με κατάλληλο bounding box γύρω από τις συντεταγμένες. Αν οι συντεταγμένες δεν έχουν συμπληρωθεί, ο χάρτης δεν εμφανίζεται, ενώ τα υπόλοιπα στοιχεία της εκδήλωσης παραμένουν διαθέσιμα.

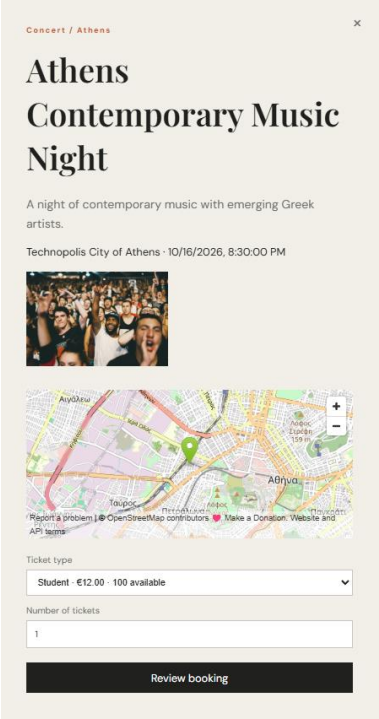


Figure 18. OpenStreetMap on Event Details

9 Σύστημα Μηνυμάτων (Messaging)

Μετά από κάθε επιτυχή κράτηση, ο συμμετέχων μπορεί να στείλει μήνυμα στον διοργανωτή της εκδήλωσης. Το σύστημα μηνυμάτων περιλαμβάνει:

- **Εισερχόμενα** (`GET /api/messages/inbox`) και **Απεσταλμένα** (`GET /api/messages/sent`), οργανωμένα σε ξεχωριστές καρτέλες στο UI.
- **Ένδειξη μη αναγνωσμένων μηνυμάτων** (`GET /api/messages/unread-count`), εμφανιζόμενη ως κόκκινο badge δίπλα στο κουμπί «Messages» στο πλοηγητικό μενού, ορατή σε κάθε σελίδα όσο ο χρήστης είναι συνδεδεμένος.
- **Διαγραφή μηνυμάτων** ανά mailbox (soft-delete μέσω `sender_deleted` / `recipient_deleted`), ώστε η διαγραφή από τον έναν χρήστη να μην επηρεάζει το αντίγραφο του άλλου.
- **Αυτόματη ειδοποίηση ακύρωσης**: όταν ο διοργανωτής ακυρώνει μια εκδήλωση, το backend δημιουργεί αυτόματα ένα μήνυμα προς κάθε χρήστη με επιβεβαιωμένη κράτηση, ενημερώνοντάς τον για την ακύρωση.

Η αποστολή μηνύματος επιτρέπεται μόνο μεταξύ διοργανωτή-συμμετέχοντα που έχουν πραγματική σχέση κράτησης για τη συγκεκριμένη εκδήλωση (έλεγχος `has_event_relationship`), ώστε να μην είναι δυνατή η αποστολή ανεπιθύμητων μηνυμάτων σε τυχαίους χρήστες.



Figure 19. Organizer Messages

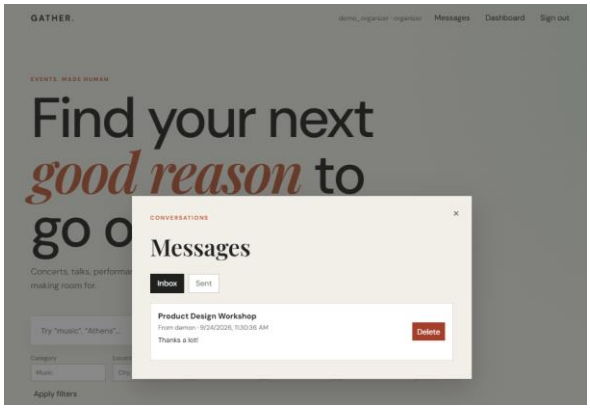


Figure 20. Organizer Review Messages

10 Εξαγωγή Δεδομένων σε XML / JSON

Ο διαχειριστής μπορεί να εξάγει όλες τις εκδηλώσεις που έχουν δημιουργηθεί από τους χρήστες:

- **XML** (`GET /api/admin/exports/events.xml`): παράγεται μέσω `xml.etree.ElementTree`, με τη σειρά στοιχείων να ακολουθεί **αυστηρά** τη σειρά του DTD της εκφώνησης (`Title`, `Category+`, `EventType`, `Venue`, `Address`, `City`, `Country`, `GeoLocation?`, `StartDateTime`, `EndDateTime`, `Capacity`, `TicketTypes`, `Bookings`, `Organizer`, `Status`, `Description`, `Media?`).
- **JSON** (`GET /api/admin/exports/events.json`): ισοδύναμη αναπαράσταση των ίδιων δεδομένων σε μορφή JSON.
- Και τα δύο endpoints επιστρέφουν το αρχείο ως download (`Content-Disposition: attachment`) και ενεργοποιούνται από κουμπιά στο dashboard του διαχειριστή, τα οποία κατεβάζουν το αρχείο απευθείας στον browser του χρήστη (μέσω `Blob` + προσωρινό `<a>` στοιχείο, ώστε να περάσει σωστά το JWT μέσω header).

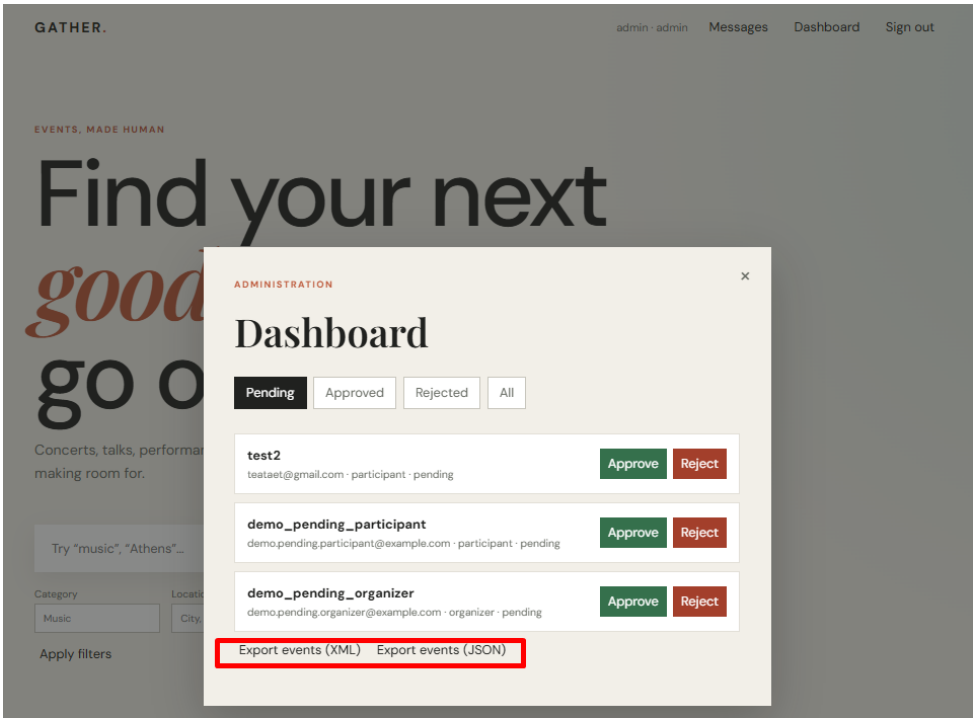


Figure 21. Export Events Buttons

```xml
<Events>
  <Event EventID="96c7fd1d-37ae-4627-8b20-9b4c6dc13fa0">
    <Title>New Title</Title>
    <Category>Music</Category>
    <EventType>Concert</EventType>
    <Venue>New Venue</Venue>
    <Address>1 Main St</Address>
    <City>Athens</City>
    <Country>Greece</Country>
    <StartDateTime>2026-07-12T20:30:00</StartDateTime>
    <EndDateTime>2026-07-12T23:00:00</EndDateTime>
    <Capacity>10</Capacity>
    <TicketTypes>
      <TicketType TicketTypeID="59770b89-dbd5-4615-bca1-832def52df78">
        <Name>General</Name>
        <Price>10.00</Price>
        <Quantity>10</Quantity>
        <Available>10</Available>
      </TicketType>
    </TicketTypes>
    <Bookings/>
    <Organizer UserID="d48f15cc-44b9-4abe-896f-02a192c7474d"/>
    <Status>draft</Status>
    <Description>desc</Description>
  </Event>
  <Event EventID="a4550a68-0844-4cab-a824-2793cdbdf0af">
    <Title>Athens Contemporary Music Night</Title>
    <Category>Music</Category>
    <EventType>Concert</EventType>
    <Venue>Technopolis City of Athens</Venue>
    <Address>Pireos 100</Address>
    <City>Athens</City>
    <Country>Greece</Country>
    <GeoLocation Latitude="37.978500" Longitude="23.711000"/>
    <StartDateTime>2026-10-16T20:30:00</StartDateTime>
    <EndDateTime>2026-10-16T23:00:00</EndDateTime>
    <Capacity>350</Capacity>
    <TicketTypes>
      <TicketType TicketTypeID="048fedee-fe1d-4c5f-a57d-727f119fe24a">
        <Name>General Admission</Name>
        <Price>18.00</Price>
        <Quantity>250</Quantity>
        <Available>248</Available>
      </TicketType>
      <TicketType TicketTypeID="0b3d131a-1fb4-4e3c-84ac-882be6190280">
        <Name>Student</Name>
        <Price>12.00</Price>
        <Quantity>100</Quantity>
        <Available>97</Available>
      </TicketType>
    </TicketTypes>
    <Bookings>
      <Booking BookingID="41c3782b-cb82-4ac7-a457-570792308bc3">
        <Attendee UserID="5079450f-96ac-4535-8079-ad29c4503949"/>
        <Time>2026-09-23T13:13:51.453364</Time>
        <TicketTypeRef>048fedee-fe1d-4c5f-a57d-727f119fe24a</TicketTypeRef>
        <NumberOfTickets>2</NumberOfTickets>
        <TotalCost>36.00</TotalCost>
        <BookingStatus>confirmed</BookingStatus>
      </Booking>
```


Figure 22. XML exported

```
{
  "Events": [
    {
      "EventID": "96c7fd1d-37ae-4627-8b20-9b4c6dc13fa0",
      "Title": "New Title",
      "Category": [
        "Music"
      ],
      "EventType": "Concert",
      "Venue": "New Venue",
      "Address": "1 Main St",
      "City": "Athens",
      "Country": "Greece",
      "GeoLocation": null,
      "StartDateTime": "2026-07-12T20:30:00",
      "EndDateTime": "2026-07-12T23:00:00",
      "Capacity": 10,
      "TicketTypes": [
        {
          "TicketTypeID": "59770b89-dbd5-4615-bca1-832def52df78",
          "Name": "General",
          "Price": "10.00",
          "Quantity": 10,
          "Available": 10
        }
      ],
      "Bookings": [],
      "Organizer": "d48f15cc-44b9-4abe-896f-02a192c7474d",
      "Status": "draft",
      "Description": "desc",
      "Media": []
    },
    {
      "EventID": "a4550a68-0844-4cab-a824-2793cdbdf0af",
      "Title": "Athens Contemporary Music Night",
      "Category": [
        "Music"
      ],
      "EventType": "Concert",
      "Venue": "Technopolis City of Athens",
      "Address": "Pireos 100",
      "City": "Athens",
      "Country": "Greece",
      "GeoLocation": {
```


Figure 23. JSON exported

11 Αλγόριθμος Συστάσεων (Biased Matrix Factorization)

Εκτός από την αναζήτηση εκδηλώσεων μέσω φίλτρων, η εφαρμογή παρέχει στους συνδεδεμένους συμμετέχοντες προσωποποιημένες προτάσεις εκδηλώσεων. Στόχος του μηχανισμού συστάσεων είναι να αξιοποιεί το προηγούμενο ιστορικό αλληλεπίδρασης των χρηστών με τις εκδηλώσεις και να εκτιμά ποιες από τις υπόλοιπες διαθέσιμες εκδηλώσεις είναι πιθανότερο να παρουσιάζουν ενδιαφέρον για κάθε χρήστη.

Η εφαρμογή δεν διαθέτει σύστημα ρητής βαθμολόγησης, όπως βαθμολογία από 1 έως 5 αστέρια. Για τον λόγο αυτό, το ενδιαφέρον του χρήστη προκύπτει έμμεσα από τις ενέργειές του στην εφαρμογή. Μια απλή προβολή μιας εκδήλωσης θεωρείται ασθενέστερη ένδειξη ενδιαφέροντος, ενώ μια επιβεβαιωμένη κράτηση θεωρείται σημαντικά ισχυρότερη ένδειξη.

Για την παραγωγή των συστάσεων υλοποιήθηκε από το μηδέν ο αλγόριθμος Biased Matrix Factorization, χωρίς χρήση έτοιμης βιβλιοθήκης μηχανικής μάθησης. Η βασική ιδέα είναι ότι τόσο οι χρήστες όσο και οι εκδηλώσεις μπορούν να αναπαρασταθούν μέσω ενός μικρού αριθμού λανθανόντων χαρακτηριστικών (latent factors). Τα χαρακτηριστικά αυτά δεν ορίζονται χειροκίνητα, αλλά μαθαίνονται αυτόματα από τα δεδομένα αλληλεπίδρασης κατά την εκπαίδευση του μοντέλου.

11.1 Θεωρητικό Υπόβαθρο

Στη Matrix Factorization θεωρούμε έναν νοητό πίνακα αλληλεπιδράσεων στον οποίο οι γραμμές αντιστοιχούν σε χρήστες και οι στήλες σε εκδηλώσεις. Κάθε γνωστή θέση του πίνακα εκφράζει την ένταση της αλληλεπίδρασης ενός χρήστη με μια εκδήλωση.

Στην πράξη ο πίνακας αυτός είναι αραιός, καθώς κάθε χρήστης αλληλεπιδρά μόνο με ένα μικρό ποσοστό του συνολικού αριθμού εκδηλώσεων. Στόχος του αλγορίθμου είναι να αξιοποιήσει τις γνωστές αλληλεπιδράσεις ώστε να εκτιμήσει ένα score για τους συνδυασμούς χρήστη–εκδήλωσης που δεν έχουν ακόμη παρατηρηθεί.

Η προβλεπόμενη τιμή για έναν χρήστη (u) και μία εκδήλωση (i) υπολογίζεται ως:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

όπου:

- (μ) είναι η μέση τιμή όλων των γνωστών αλληλεπιδράσεων και αποτελεί το βασικό επίπεδο αναφοράς του μοντέλου,
- (b_u) είναι το bias του χρήστη και εκφράζει κατά πόσο ο συγκεκριμένος χρήστης παρουσιάζει συνολικά μεγαλύτερη ή μικρότερη ένταση αλληλεπίδρασης σε σχέση με τον μέσο όρο,
- (b_i) είναι το bias της εκδήλωσης και αποτυπώνει αν μία εκδήλωση εμφανίζει γενικά μεγαλύτερο ή μικρότερο ενδιαφέρον σε σχέση με τις υπόλοιπες,
- ($\mathbf{p}_u$) είναι το διάνυσμα λανθανόντων χαρακτηριστικών του χρήστη,

- (\mathbf{q}_i) είναι το αντίστοιχο διάνυσμα λανθανόντων χαρακτηριστικών της εκδήλωσης.

Το εσωτερικό γινόμενο (\mathbf{p}_u^\top\mathbf{q}_i) εκφράζει τον βαθμό συμβατότητας μεταξύ των λανθανόντων χαρακτηριστικών ενός χρήστη και μιας εκδήλωσης. Για παράδειγμα, ο αλγόριθμος μπορεί μέσα από τα δεδομένα να μάθει ότι συγκεκριμένες ομάδες χρηστών παρουσιάζουν παρόμοιο ενδιαφέρον για συγκεκριμένες ομάδες εκδηλώσεων, χωρίς να απαιτείται τα χαρακτηριστικά αυτά να έχουν οριστεί εκ των προτέρων ως «μουσική», «αθλητισμός» ή κάποια άλλη συγκεκριμένη κατηγορία.

Στην παρούσα υλοποίηση χρησιμοποιούνται οκτώ λανθάνουσες διαστάσεις:

mathbf{p}_u,\mathbf{q}_i \in \mathbb{R}^{8}

Το αποτέλεσμα (\hat r_{ui}) δεν ερμηνεύεται ως πιθανότητα ούτε ως πραγματική βαθμολογία που έχει δώσει ο χρήστης. Χρησιμοποιείται ως συγκριτικό score για την ταξινόμηση των υποψήφιων εκδηλώσεων. Όσο υψηλότερο είναι το score μιας εκδήλωσης για τον συγκεκριμένο χρήστη, τόσο υψηλότερα μπορεί να τοποθετηθεί στη λίστα συστάσεων.

Για κάθε γνωστή αλληλεπίδραση υπολογίζεται αρχικά το σφάλμα μεταξύ της πραγματικής τιμής (r_{ui}) και της προβλεπόμενης τιμής:

e_{ui}=r_{ui}-\hat r_{ui}

Στη συνέχεια οι παράμετροι του μοντέλου μεταβάλλονται σταδιακά με Stochastic Gradient Descent (SGD), έτσι ώστε σε κάθε επανάληψη η πρόβλεψη να πλησιάζει περισσότερο την πραγματική τιμή.

Οι ενημερώσεις των biases πραγματοποιούνται σύμφωνα με:

b_u \leftarrow b_u+\eta(e_{ui}-\lambda b_u)

b_i \leftarrow b_i+\eta(e_{ui}-\lambda b_i)

ενώ τα latent vectors ενημερώνονται με:

\mathbf{p}_u \leftarrow
\mathbf{p}u+\eta(e{ui}\mathbf{q}_i-\lambda\mathbf{p}_u)

\mathbf{q}_i \leftarrow
\mathbf{q}i+\eta(e{ui}\mathbf{p}_u-\lambda\mathbf{q}_i)

Η παράμετρος (\eta) αποτελεί τον ρυθμό εκμάθησης και καθορίζει το μέγεθος της μεταβολής των παραμέτρων σε κάθε βήμα. Η παράμετρος (\lambda) χρησιμοποιείται για regularization, περιορίζοντας την ανεξέλεγκτη αύξηση των παραμέτρων και μειώνοντας τον κίνδυνο υπερπροσαρμογής στα δεδομένα εκπαίδευσης.

Στην υλοποίηση χρησιμοποιούνται 8 latent factors, learning rate (\eta=0.02), regularization (\lambda=0.05) και 40 εποχές εκπαίδευσης. Η σχετική λογική βρίσκεται στο app/services/recommendations.py.

11.2 Σήμα Αλληλεπίδρασης (Implicit Feedback)

Η εφαρμογή δεν ζητά από τους χρήστες να βαθμολογούν τις εκδηλώσεις. Κατά συνέπεια, για την εκπαίδευση του μοντέλου χρησιμοποιείται implicit feedback, δηλαδή έμμεσες ενδείξεις ενδιαφέροντος που προκύπτουν από τις ενέργειες του χρήστη.

Ενέργεια χρήστη	Βάρος
Προβολή εκδήλωσης (`EventView`)	1.0
Επιβεβαιωμένη κράτηση (`Booking`, status `CONFIRMED`)	5.0

Η διαφοροποίηση των δύο βαρών χρησιμοποιείται ώστε μία κράτηση να θεωρείται ισχυρότερο σήμα προτίμησης από μία απλή προβολή. Η προβολή δείχνει ότι μία εκδήλωση τράβηξε την προσοχή του χρήστη, ενώ η κράτηση αποτελεί πιο ισχυρή ένδειξη ότι η συγκεκριμένη εκδήλωση ανταποκρίνεται στα ενδιαφέροντά του.

Κατά τη δημιουργία της τελικής λίστας συστάσεων δεν είναι χρήσιμο να εμφανίζονται ξανά αντικείμενα με τα οποία ο χρήστης έχει ήδη αλληλεπιδράσει. Για τον λόγο αυτό, εκδηλώσεις τις οποίες ο χρήστης έχει ήδη προβάλει ή για τις οποίες έχει πραγματοποιήσει κράτηση εξαιρούνται από τις υποψήφιες προτάσεις. Ο αλγόριθμος χρησιμοποιείται επομένως για να κατατάξει τις υπόλοιπες διαθέσιμες εκδηλώσεις.

11.3 Cold-Start (Χρήστης Χωρίς Ιστορικό Κρατήσεων)

Ένα συνηθισμένο πρόβλημα στα συστήματα συστάσεων είναι το λεγόμενο cold-start: η περίπτωση κατά την οποία δεν υπάρχουν αρκετά δεδομένα για έναν νέο χρήστη ώστε να παραχθούν αξιόπιστες προσωποποιημένες συστάσεις.

Στην εφαρμογή διακρίνονται δύο διαφορετικές περιπτώσεις.

Στην πρώτη περίπτωση, ο χρήστης δεν έχει πραγματοποιήσει ακόμη καμία κράτηση, αλλά έχει ήδη επισκεφθεί σελίδες εκδηλώσεων. Σύμφωνα με την απαίτηση της εκφώνησης, οι διαθέσιμες προβολές χρησιμοποιούνται ως το ιστορικό αλληλεπίδρασης του συγκεκριμένου χρήστη. Με αυτόν τον τρόπο είναι δυνατόν να παραχθεί κάποιος βαθμός προσωποποίησης ακόμη και πριν από την πρώτη κράτηση.

Στη δεύτερη περίπτωση, ένας εντελώς νέος χρήστης δεν διαθέτει ούτε κρατήσεις ούτε προβολές. Στην περίπτωση αυτή δεν υπάρχει προσωπικό σήμα από το οποίο να μπορεί να εξαχθεί προτίμηση. Για τον λόγο αυτό ενεργοποιείται fallback μηχανισμός και επιστρέφεται λίστα δημοφιλών ή επερχόμενων

δημοσιευμένων εκδηλώσεων. Έτσι η σελίδα συστάσεων παραμένει χρήσιμη ακόμη και κατά την πρώτη είσοδο του χρήστη στην εφαρμογή.

Η fallback συμπεριφορά δεν αποτελεί προσωποποιημένη πρόβλεψη του Matrix Factorization, αλλά εναλλακτική στρατηγική η οποία χρησιμοποιείται αποκλειστικά όταν δεν υπάρχουν επαρκή δεδομένα για τον χρήστη.

11.4 Έλεγχος (Testing) του Αλγορίθμου

Ο μηχανισμός συστάσεων ελέγχθηκε μέσω αυτοματοποιημένων tests στο tests/test_recommendations.py. Οι δοκιμές επικεντρώνονται κυρίως στην ορθότητα της συμπεριφοράς του συστήματος και όχι μόνο στην εκτέλεση των μαθηματικών υπολογισμών.
Ελέγχονται συγκεκριμένα οι ακόλουθες περιπτώσεις:

- όταν δεν υπάρχουν δημοσιευμένες εκδηλώσεις, δεν επιστρέφονται συστάσεις,
- για χρήστη χωρίς κανένα ιστορικό ενεργοποιείται σωστά ο fallback μηχανισμός,
- εκδηλώσεις για τις οποίες έχει ήδη πραγματοποιηθεί κράτηση δεν προτείνονται εκ νέου,
- εκδηλώσεις τις οποίες έχει ήδη προβάλει ο χρήστης εξαιρούνται επίσης από τις συστάσεις,
- προβολή μη δημοσιευμένης εκδήλωσης δεν καταγράφεται ως έγκυρη αλληλεπίδραση.

Οι παραπάνω έλεγχοι είναι σημαντικοί επειδή εξετάζουν όχι μόνο αν ο αλγόριθμος παράγει αριθμητικά scores, αλλά και αν το συνολικό recommender system ακολουθεί τους επιχειρησιακούς κανόνες της εφαρμογής.

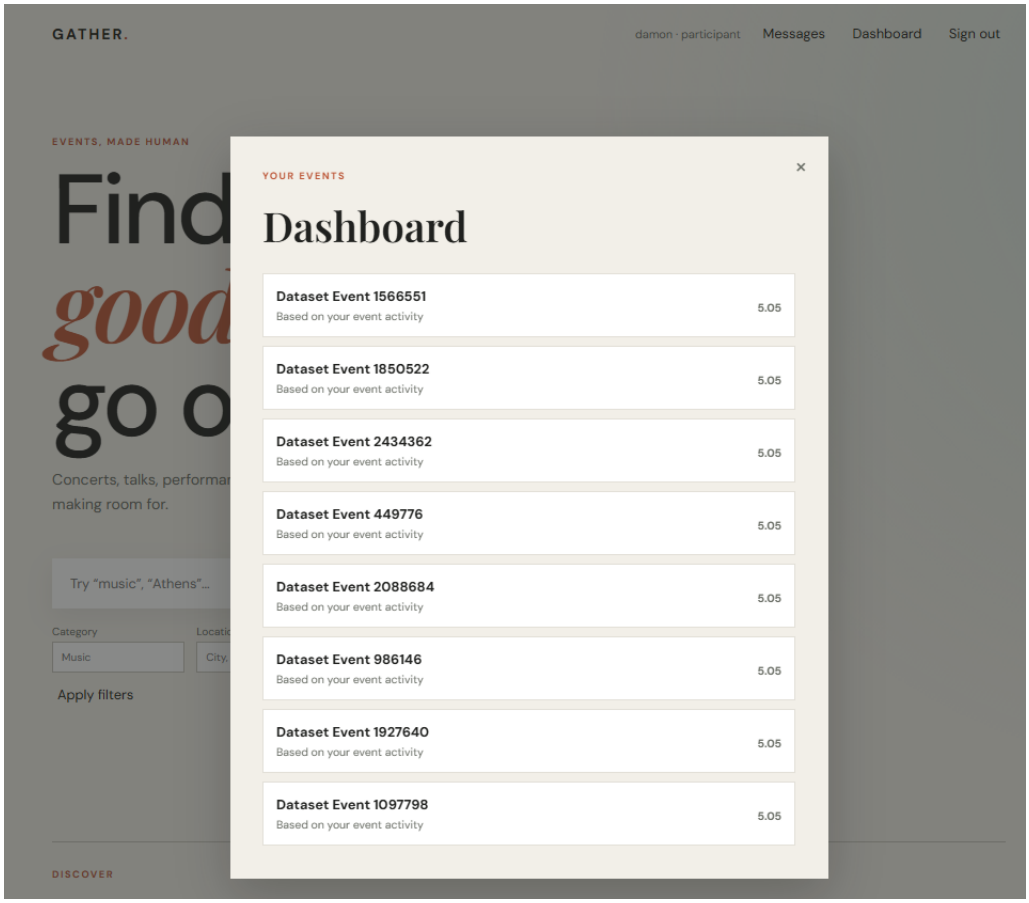


Figure 24. Events Recommendations

Για τη δημιουργία μεγαλύτερου όγκου δεδομένων και τη δοκιμή του μηχανισμού συστάσεων χρησιμοποιείται ο seeder app/scripts/seed_dataset.py.
Επειδή τα αρχικά CSV αρχεία περιλαμβάνουν πολύ μεγάλο αριθμό εγγραφών, τα δεδομένα δεν εισάγονται αυτόματα σε κάθε εκκίνηση της εφαρμογής. Αντίθετα, η εισαγωγή πραγματοποιείται ελεγχόμενα, με δυνατότητα περιορισμού του αριθμού χρηστών και εκδηλώσεων.
Ενδεικτικά, με την εντολή:
docker compose exec backend python -m app.scripts.seed_dataset --events 300 --users 100
εισάγονται έως 300 εκδηλώσεις και 100 χρήστες, μαζί με προβολές και περιορισμένο αριθμό επιβεβαιωμένων κρατήσεων.
Η διαδικασία είναι idempotent, επομένως η επανάληψη της ίδιας εντολής δεν δημιουργεί διπλότυπες εγγραφές. Σε δοκιμαστική εκτέλεση με 300 events και 100 users δημιουργήθηκαν 244 interactions και το σύστημα επέστρεψε 8 recommendations για τον αντίστοιχο seeded χρήστη.

12 Frontend – Οθόνες και Ροές Χρήστη

Το frontend υλοποιήθηκε ως **Single Page Application (SPA)** με React + TypeScript, χρησιμοποιώντας modals για τις δευτερεύουσες οθόνες (login/εγγραφή, λεπτομέρειες εκδήλωσης, dashboard, μηνύματα κ.λπ.) πάνω από μία κεντρική σελίδα ανακάλυψης εκδηλώσεων. Η επιλογή αυτή αξιοποιεί την ελευθερία σχεδίασης που δίνει ρητά η εκφώνηση.

Οθόνη	Περιγραφή
Welcome / Home	Κεντρική σελίδα με μπάρα αναζήτησης, φίλτρα και πλέγμα δημοσιευμένων εκδηλώσεων. Διαθέσιμη σε επισκέπτες χωρίς σύνδεση.
Sign in / Create account	Modal φόρμας login και εγγραφής με πλήρη επικύρωση πεδίων.
Λεπτομέρειες Εκδήλωσης	Τίτλος, περιγραφή, φωτογραφίες (gallery), χάρτης OpenStreetMap, επιλογή τύπου εισιτηρίου/ποσότητας, διβάθμια επιβεβαίωση κράτησης, φόρμα επικοινωνίας με τον διοργανωτή.
Dashboard Διαχειριστή	Καρτέλες Pending/Approved/Rejected/All, λεπτομέρειες χρήστη, εγκρίσεις/απορρίψεις, εξαγωγή XML/JSON.
Dashboard Διοργανωτή	Λίστα ιδίων εκδηλώσεων, δημιουργία νέας εκδήλωσης, επεξεργασία, δημοσίευση, ακύρωση, διαγραφή, προβολή κρατήσεων ανά εκδήλωση.
Dashboard Συμμετέχοντα	Προσωποποιημένες συστάσεις εκδηλώσεων.
Messages	Καρτέλες Inbox/Sent, ένδειξη μη αναγνωσμένων, διαγραφή μηνυμάτων.

Figure 25. Frontend Views

Το κύριο αρχείο υλοποίησης είναι το [`frontend/src/App.tsx`](../frontend/src/App.tsx), με τη μορφοποίηση στο [`frontend/src/styles.css`](../frontend/src/styles.css).

13 Ασφάλεια Εφαρμογής

Στην υλοποίηση εφαρμόστηκαν βασικές πρακτικές ασφαλούς ανάπτυξης, με γνώμονα κινδύνους που περιγράφονται και στο OWASP Top 10:

- **Κρυπτογράφηση κωδικών**: Argon2id (μνήμη-εντατικό hashing, ανθεκτικό σε επιθέσεις GPU/ASIC).
- **Χωρίς αποθήκευση κωδικών σε καθαρό κείμενο** πουθενά (ούτε logs).
- **JWT με χρόνο λήξης** (30 λεπτά) αντί για μόνιμα tokens.
- **RBAC σε επίπεδο endpoint** μέσω FastAPI dependencies — κάθε ενέργεια ελέγχεται ξεχωριστά, όχι μόνο στο UI.
- **Επικύρωση εισόδου** σε όλα τα endpoints μέσω Pydantic (τύποι, όρια μήκους, ελάχιστες τιμές, μοναδικότητα username/email).
- **CORS** περιορισμένο σε συγκεκριμένα origins (`http://localhost:5173` για ανάπτυξη, `https://localhost` για production build).
- **SQL Injection**: αποτρέπεται εγγενώς μέσω της χρήσης SQLAlchemy ORM/Core με παραμετροποιημένα ερωτήματα — δεν κατασκευάζεται ποτέ SQL με string concatenation.
- **Εξουσιοδότηση σε επίπεδο αντικειμένου**: π.χ. ένας διοργανωτής δεν μπορεί να δει/επεξεργαστεί εκδηλώσεις άλλου διοργανωτή (έλεγχος `organizer_id` σε κάθε ερώτημα).
- **HTTPS/TLS σε production**: μέσω Caddy reverse proxy, με τερματισμό TLS πριν την είσοδο στο backend

14 Εγκατάσταση και Εκτέλεση

14.1 Ανάπτυξη (development) χωρίς Docker

Backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -e '.[dev]'
uvicorn app.main:app --reload
```

Το API είναι διαθέσιμο στο `http://127.0.0.1:8000`, με τεκμηρίωση στο `http://127.0.0.1:8000/docs`. Ενσωματωμένος λογαριασμός διαχειριστή: `admin` / `admin123!` (να αλλαχθεί υποχρεωτικά πριν από οποιαδήποτε πραγματική ανάπτυξη, μέσω μεταβλητών περιβάλλοντος στο `.env`).

Frontend:

```
cd frontend
npm install
npm run dev
```

Διαθέσιμο στο `http://localhost:5173`, καλώντας το backend στο `http://localhost:8000/api`.

14.2 Εκτέλεση μέσω Docker Compose (HTTPS)

```
docker compose up -d --build
```

Ξεκινούν τρεις υπηρεσίες:

- **postgres** — η βάση δεδομένων.
- **backend** — εκτελεί αυτόματα `alembic upgrade head` και σερβίρει το API στο `http://localhost:8000`.
- **web** — παράγει το production build του frontend και το σερβίρει μέσω **Caddy** στο `https://localhost`, προωθώντας τα αιτήματα `/api/*` στο backend. Το πιστοποιητικό είναι αυτο-υπογεγραμμένο (self-signed) — ο browser θα εμφανίσει προειδοποίηση που πρέπει να γίνει αποδεκτή χειροκίνητα, κάτι φυσιολογικό σε τοπικό περιβάλλον ανάπτυξης/επίδειξης. Σε πραγματική ανάπτυξη με δημόσιο domain, αρκεί η αντικατάσταση της στατικής διαδρομής πιστοποιητικού στο `frontend/Caddyfile` με `tls internal` ή η πλήρης αφαίρεση της οδηγίας `tls`, ώστε ο Caddy να αποκτήσει αυτόματα έγκυρο πιστοποιητικό μέσω ACME/Let's Encrypt.

Η προειδοποίηση `ERR_CERT_AUTHORITY_INVALID` δεν σημαίνει ότι η σύνδεση δεν χρησιμοποιεί HTTPS. Σημαίνει ότι το πιστοποιητικό εκδόθηκε τοπικά και δεν έχει υπογραφεί από δημόσια, έμπιστη Certificate Authority. Συνεπώς, κατά την τοπική επίδειξη ο χρήστης μπορεί να επιλέξει **Advanced → Proceed to localhost**. Για απλή ανάπτυξη χωρίς προειδοποίηση διατίθεται επίσης το `http://localhost` στο port 80· η επίσημη ασφαλής διαδρομή της εφαρμογής παραμένει το `https://localhost` στο port 443.

Τερματισμός: `docker compose down` (με `-v` για πλήρη διαγραφή δεδομένων PostgreSQL).

14.3 Demo Δεδομένα και Λογαριασμοί Επίδειξης

Για να είναι δυνατή η άμεση δοκιμή όλων των λειτουργιών της εφαρμογής, το Docker Compose ενεργοποιεί τη μεταβλητή `SEED_DEMO_DATA=true`. Κατά την εκκίνηση δημιουργούνται αυτόματα:

- τρεις δημοσιευμένες εκδηλώσεις σε πραγματικές τοποθεσίες της Αθήνας,
- διαφορετικές κατηγορίες και τύποι εισιτηρίων,
- γεωγραφικές συντεταγμένες για την προβολή OpenStreetMap,
- φωτογραφίες/media URLs,
- μία ενδεικτική κράτηση για την πρώτη εκδήλωση,
- ένας εγκεκριμένος διοργανωτής και ένας εγκεκριμένος συμμετέχων.

Οι λογαριασμοί επίδειξης είναι:

Table 6. Λογαριασμοί Επίδειξης

Ρόλος	Username	Password
Διαχειριστής	`admin`	`admin123!`
Διοργανωτής	`demo_organizer`	`organizer123!`
Συμμετέχων	`demo_participant`	`participant123!`

Pending λογαριασμοί για δοκιμή του Admin Dashboard:

Table 7. Pending Λογαριασμοί

Αιτούμενος ρόλος	Username	Password	Αρχική κατάσταση
Διοργανωτής	`demo_pending_organizer`	`pendingorg123!`	`PENDING`
Συμμετέχων	`demo_pending_participant`	`pendingpar123!`	`PENDING`

Το seed είναι **idempotent**: σε κάθε επανεκκίνηση ελέγχει αν τα δεδομένα υπάρχουν και δεν δημιουργεί διπλούς χρήστες ή εκδηλώσεις. Τα δεδομένα αποθηκεύονται στο volume `postgres_data`, επομένως παραμένουν μετά από `docker compose down`. Αν χρησιμοποιηθεί `docker compose down -v`, το volume και η βάση διαγράφονται· στην επόμενη εκκίνηση τα demo δεδομένα δημιουργούνται ξανά από την αρχή.

Εκτός Docker, το seed είναι απενεργοποιημένο από προεπιλογή μέσω `SEED_DEMO_DATA=false`. Έτσι τα demo δεδομένα χρησιμοποιούνται μόνο για την τοπική επίδειξη και δεν εισάγονται αυτόματα σε κανονικό περιβάλλον παραγωγής.

15 Παραδοχές και Σχεδιαστικές Αποφάσεις

Ρόλος κατά την εγγραφή: Ο χρήστης επιλέγει κατά την εγγραφή αν επιθυμεί ρόλο Συμμετέχοντα ή Διοργανωτή· ο ρόλος Διαχειριστή δεν είναι διαθέσιμος μέσω δημόσιας εγγραφής (μόνο ενσωματωμένος λογαριασμός), για προφανείς λόγους ασφαλείας.
Επισκέπτες: Η δημόσια αναζήτηση/προβολή εκδηλώσεων δεν απαιτεί JWT, ώστε να ικανοποιείται η ρητή απαίτηση πρόσβασης επισκέπτη· όλες οι υπόλοιπες ενέργειες απαιτούν πιστοποίηση.
Τύπος κράτησης: Οι κρατήσεις αφορούν ποσότητα ανά τύπο εισιτηρίου (όχι συγκεκριμένη αριθμημένη θέση), καθώς η εκφώνηση δεν απαιτεί ρητά επιλογή θέσης..
Διαγραφή μηνυμάτων: Υλοποιήθηκε ως «μαλακή» διαγραφή (soft-delete) ανά mailbox, ώστε το ιστορικό του άλλου συνομιλητή να διατηρείται.
HTTPS με αυτο-υπογεγραμμένο πιστοποιητικό: Επιλέχθηκε στατικό πιστοποιητικό (αντί για την αυτόματη εσωτερική CA του Caddy) λόγω τεχνικού περιορισμού στο περιβάλλον ανάπτυξης/δοκιμών (βλ. σημείωση παρακάτω)· σε κανονικό Docker host, η αυτόματη έκδοση πιστοποιητικών του Caddy λειτουργεί κανονικά.
Βάση δεδομένων ανάπτυξης: Για γρήγορη τοπική ανάπτυξη χωρίς Docker χρησιμοποιείται SQLite ως προεπιλογή, ενώ η «πραγματική» βάση-στόχος είναι η PostgreSQL (χρησιμοποιούμενη στο Docker Compose και ελεγμένη με πραγματικά migrations/tests συγχρονισμού).

16 Επίλογος

Η παρούσα εργασία είχε ως στόχο την ανάπτυξη μιας ολοκληρωμένης διαδικτυακής εφαρμογής διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων, συνδυάζοντας λειτουργίες διαχείρισης χρηστών, εκδηλώσεων και κρατήσεων με μηχανισμούς επικοινωνίας, εξαγωγής δεδομένων και προσωποποιημένων συστάσεων. Η τελική υλοποίηση οργανώθηκε σε διακριτά επίπεδα frontend, backend και βάσης δεδομένων, γεγονός που διευκολύνει τόσο τη συντήρηση όσο και την περαιτέρω επέκταση του συστήματος.

Ιδιαίτερα σημαντικό σημείο της υλοποίησης αποτέλεσε η διαχείριση της ταυτόχρονης πρόσβασης κατά την κράτηση εισιτηρίων. Αντί για έναν απλό μηχανισμό ελέγχου της διαθεσιμότητας πριν από την ενημέρωση, χρησιμοποιήθηκε ατομική ενημέρωση της βάσης δεδομένων μέσα σε συναλλαγή. Με τον τρόπο αυτό αποφεύγεται το φαινόμενο race condition, κατά το οποίο δύο ταυτόχρονα αιτήματα θα μπορούσαν διαφορετικά να δεσμεύσουν τις ίδιες τελευταίες διαθέσιμες θέσεις. Η ορθότητα της συγκεκριμένης προσέγγισης επιβεβαιώθηκε και μέσω αυτοματοποιημένης δοκιμής με ταυτόχρονες αιτήσεις κράτησης.

Ένα ακόμη σημαντικό τμήμα της εργασίας ήταν η ανάπτυξη του μηχανισμού προσωποποιημένων συστάσεων. Ο αλγόριθμος Biased Matrix Factorization υλοποιήθηκε χωρίς τη χρήση έτοιμης βιβλιοθήκης μηχανικής μάθησης και προσαρμόστηκε σε δεδομένα έμμεσης ανατροφοδότησης, αξιοποιώντας τις προβολές εκδηλώσεων και τις επιβεβαιωμένες κρατήσεις ως ενδείξεις διαφορετικής έντασης ενδιαφέροντος. Παράλληλα, προβλέφθηκε ειδική συμπεριφορά για την περίπτωση νέων χρηστών με περιορισμένο ή μηδενικό ιστορικό αλληλεπίδρασης.

Σημαντική ήταν επίσης η εφαρμογή μηχανισμών ασφάλειας σε διαφορετικά επίπεδα του συστήματος. Οι κωδικοί αποθηκεύονται με Argon2id hashing, η αυθεντικοποίηση πραγματοποιείται μέσω JWT, οι ενέργειες προστατεύονται με RBAC και ελέγχους ιδιοκτησίας αντικειμένων, ενώ στο περιβάλλον Docker η πρόσβαση στην εφαρμογή πραγματοποιείται μέσω HTTPS. Η χρήση SQLAlchemy και Pydantic συνεισφέρει επιπλέον στην ασφαλή πρόσβαση στη βάση δεδομένων και στην επικύρωση των δεδομένων εισόδου.

Η ανάπτυξη της εφαρμογής ανέδειξε στην πράξη τη σημασία της σωστής σχεδίασης τόσο της επιχειρησιακής λογικής όσο και της αρχιτεκτονικής ενός πολυεπίπεδου web συστήματος. Η χρήση PostgreSQL, Alembic migrations, Docker Compose και αυτοματοποιημένων tests προσφέρει ένα επαναλήψιμο και ελεγχόμενο περιβάλλον εγκατάστασης και δοκιμών, ενώ ο διαχωρισμός του REST API από το React frontend καθιστά τα επιμέρους τμήματα του συστήματος περισσότερο ανεξάρτητα και επεκτάσιμα.

Ως πιθανές μελλοντικές επεκτάσεις θα μπορούσαν να εξεταστούν η ενσωμάτωση πραγματικού μηχανισμού ηλεκτρονικών πληρωμών, η υποστήριξη αριθμημένων θέσεων, η χρήση δημόσιου πιστοποιητικού TLS σε περιβάλλον παραγωγής και η περαιτέρω αξιολόγηση του συστήματος συστάσεων με μεγαλύτερο όγκο πραγματικών αλληλεπιδράσεων και κατάλληλες μετρικές αξιολόγησης. Οι επεκτάσεις αυτές δεν αποτελούν μέρος της παρούσας υλοποίησης, αλλά μπορούν να βασιστούν στην υπάρχουσα αρχιτεκτονική χωρίς να απαιτείται ουσιαστικός επανασχεδιασμός του συστήματος.